Formal Verification of the Lightning Network in Tamarin

***Abstract*—The Lightning Network routes payments over chains of payment channels using hash time-locked contracts (HTLCs). We present one Tamarin theory covering the channel layer (unboundedly many updates, revocation, punishment) and multi-hop routing over routes of any length, against a Dolev-Yao adversary who can also compromise long-term keys. We prove that only cheating or compromised parties are punished, that settlements are authenticated by the receiver's invoice, and that preimages, which our model never sends on the network, stay secret. Under three liveness assumptions, imposed as restrictions that remove every run violating them, an uncompromised intermediary never has its outgoing HTLC redeemed while its incoming one is refunded, and payment atomicity holds in runs without key compromise. The assumption that intermediaries claim in time is necessary: without it, Tamarin finds a run without compromise in which an intermediary loses funds. We give the wormhole attack as a Tamarin-checked trace: colluding intermediaries bypass an honest one, which is refunded but earns no fee. Conversely, we prove for routes of any length that a hop of a settled payment whose hash is offered once can be bypassed only if some long-term key is compromised. On a fixed route, per-hop locks in the style of point time-locked contracts (PTLCs) prevent the wormhole around an honest intermediary when the sender is honest, whereas plain HTLCs admit it. All results are machine-checked with Tamarin 1.10 and reproduced with one command.**

***Index Terms*—Lightning Network, payment channels, HTLC, wormhole attack, symbolic verification, Tamarin**

I. Introduction

Bitcoin [1] is the most important cryptocurrency in terms of market capitalisation. It is, however, lacking in many usability and scalability aspects: it is costly, has low bandwidth (only a few dozen transactions per second), and has high latency. Many protocols strive to alleviate those shortcomings.

a) Payment Channel Networks: Notably, payment channels enable a pair of users to quickly and cheaply perform arbitrarily many payments by relying on the blockchain only to resolve disputes. Both agents pool together funds and then iteratively update their balance. The scripts involved and the protocol itself ensure that no honest agent loses money, by relying on a revocation mechanism: any party closing a channel using an outdated favourable state can be punished, consequently losing all funds invested in the channel. However, payment channels are constrained to pairs of users. Two unconnected users still need to establish a channel in order to perform a payment; this is often not cost effective. Thus arose the concept of payment channel networks: one transitively propagates funds across payment channels to perform a payment between two users that are part of the same network but not connected directly. The most popular of such protocols is perhaps the Lightning Network [2], with about $170M locked on the network [3].

Due to the sheer complexity of its specification [4], it suffers from many attacks [5], [6], [7]. And despite promising ongoing efforts [8], [9], it still lacks a machine-checked model that covers both the channel layer and multi-hop routing, for unboundedly many channels, updates and payments and for routes of any length. This paper attempts to alleviate that issue.

b) Protocol Verification: Designing and analysing cryptographic protocols is a notoriously difficult and error-prone task, as already illustrated by the various attacks and tentative fixes on the Lightning Network. Formal methods provide hope for a more robust verification of protocols, and for automation.

Protocol verification is further split between two main classes of models: the computational model, inherited from cryptography, where the attackers and the participants are polynomial-time algorithms and results are typically probabilistic; and the symbolic (or *Dolev-Yao*) model [10], where the attacker has infinite computational power but is limited to a fixed term algebra. The symbolic model notably abstracts away from cryptography and focuses on the logic of a protocol, where much of the attacker activity is typically found.

Although some tools exist to help the cryptographer in the computational model, such as CryptoVerif [11], EasyCrypt [12], Squirrel [13] or CryptoVampire [14], they are all brittle, hard to use or narrow in scope. In contrast, symbolic verification enjoys very successful automated verifiers in the form of ProVerif [15] and Tamarin [16]. Mechanised proofs now exist for widely used protocols such as WireGuard [17] and TLS 1.3 [18], among many others. In particular, Tamarin is a multiset rewriting-based automated protocol verifier with a large pedigree of verified protocols, TLS 1.3 among them. It proves trace properties for an unbounded number of sessions and states assumptions explicitly, as restrictions on traces.

c) Contributions: All results are checked with Tamarin 1.10. The all-traces statements hold for every trace of the model that satisfies its restrictions (Sec. IV), that is, for any number of parties, channels, channel updates

and payments.

- *One theory, one adversary.* A single TAMARIN theory (`lightning.spthy`) covers the channel layer (opening, unboundedly many updates, revocation, cooperative and unilateral closing, punishment) and multi-hop HTLC routing, where one generic rule, `Forward_HTLC`, handles every hop of routes of any length. A Dolev-Yao adversary can also compromise any long-term key at any time, and two compromised parties on the route of a payment can relay its preimage around the parties between them (`AttackerSkip`; Sec. IV-A). The layers share the channel topology and a per-channel slot. Their contract, that each HTLC is resolved at most once, is the named restriction `OneResolutionPerHTLC`, which we justify informally; we claim no composition theorem (Sec. IV).
- *Safety.* We prove 32 all-traces safety lemmas. Among them: a party is punished only if it published a revoked state or its key was compromised; a sender settles only against an invoice of the receiver, unless the receiver's key was compromised; preimages stay secret (our model never sends them); no HTLC is both redeemed and refunded. Under three named liveness restrictions, T1 to T3, an intermediary whose outgoing HTLC is redeemed has its incoming HTLC refunded only if its key was compromised before that HTLC timed out, and payment atomicity holds in runs without key compromise. These restrictions remove every trace that violates them, including traces in which an uncompromised forwarder could not have acted in time; the results say nothing about such traces. Ten exists-trace lemmas show that the protocol runs and that attacks with compromised keys, such as the wormhole below, are reachable (Sec. V).
- *The wormhole, as a trace and as a theorem.* TAMARIN checks (by replaying a saved proof) a five-party trace in which the payment settles while an honest intermediary is refunded on its outgoing HTLC and earns no fee (Sec. V-E). Conversely, if the sender settles, a forwarder on its route is refunded on its outgoing HTLC only if some long-term key is compromised. For each of the first six hops of a route of any length, we prove this under the liveness restrictions; for every hop, we prove it without them (so it also holds with them), provided the payment hash is offered only once. Neither result says whose key is compromised, or when (Sec. VI).
- *Necessity of the liveness assumptions.* Without T3 (`IntermediaryMustClaim`), TAMARIN finds a trace without key compromise in which an intermediary's outgoing HTLC is redeemed while its incoming HTLC is refunded, so T3 is necessary for intermediary safety. Without T1, all five liveness lemmas we check remain provable; without T2, both intermediary-safety lemmas do. The CLTV-delta ordering of timeouts (`ForwardTimeGap`) is a lemma derived from T2 (Sec. V-D).
- *Countermeasure.* On a fixed route of three intermediaries, per-hop locks derived with secret tweaks, a symbolic stand-in for point time-locked contracts (PTLCs) [6], prevent bypassing the uncompromised middle hop unless the sender is compromised; with a common payment hash, the bypass is reachable (Sec. VI-E).
- *Reproducibility.* One command (`make all`) reproduces all results: the 55 lemmas of the main model, the 24 of the any-length theorem, the 21 of the auxiliary models, and the necessity runs (Sec. V-B).

Unlike the TLA+ model checking of [8], which explores finite instances, our all-traces results hold for unboundedly many channels, updates and payments, and for routes of any length.

Sec. II recalls the protocol, Secs. III and IV present the symbolic model and our methodology, Secs. V and VI our results, and Secs. VII to IX related work, limitations and conclusions.

II. Protocol Overview and Threat Model

The Lightning Network can be split into two main parts: the *Payment Channels* and the *Routing*. The first one enables transferring funds quickly and efficiently between two participants with reduced interaction with a blockchain. However, the setup is comparatively expensive. The routing protocol organises and secures transfers going beyond a single channel, scaling localised pairwise payments to a whole network.

Each part carries its own cryptographic mechanism. In the payment channel, revocation keys protect against *outdated states*. In the routing protocol, a payment hash and a deadline control the *transfer*. They meet only in the commitment transaction, where an in-flight payment is an extra output. It is subject to the same revocation as the rest of the channel. In our model, this meeting point becomes an explicit interface between the two layers (Sec. IV). This section gives an overview of these two parts.

A. Payment Channels

The Payment Channel protocol fundamentally enables two agents to *commit* and *revoke* a shared blockchain-related "state". This state typically tracks the balance of some multi-sig [19] *common fund*, effectively transferring coins between its two participants. The state is represented by valid transactions spending that common fund, and a payment channel ensures these transactions are valid and that no agent can publish an old, favourable state. The latter property comes from careful signing (restricting transactions to agreed-upon shapes) and *punishing*: a participant may unconditionally retrieve *all* funds locked in a channel if their counterparty puts an old state on chain.

In the context of the Lightning Network, the state also tracks in-flight payments which will propagate further than

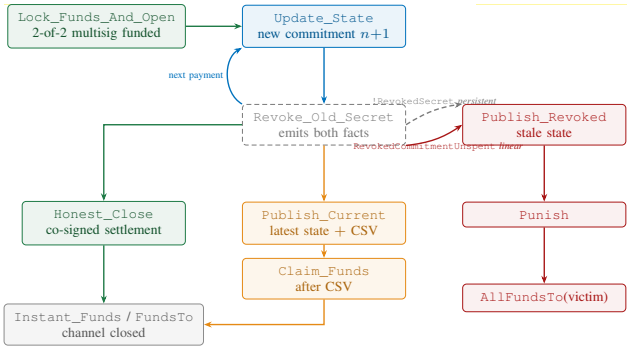


Fig. 1. Channel lifecycle, labelled with the rules of our model. Green: opening and cooperative close; blue: update loop; orange: unilateral close with its CSV delay; red: revoked publish and punishment. `Revoke_Old_Secret` emits *two* facts, and both gate punishment: the persistent `!RevokedSecret` (dashed, checked but never consumed) and the linear `RevokedCommitmentUnspent` (solid, consumed once); gating on the persistent fact alone would let the same revoked commitment be punished more than once (Sec. V-B). Cooperative and unilateral closes end in settlement; punishment ends in `AllFundsTo`. Grace-period and pending-publish paths, and the counterparty's instant output on a unilateral close, are elided.

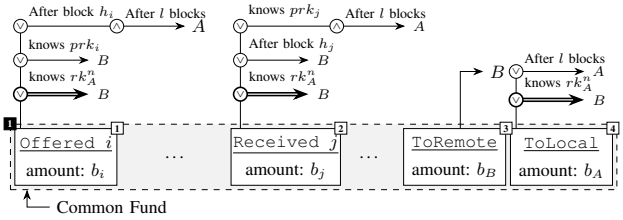


Fig. 2. Commitment transaction held by A. The CSV is set to l, and the timeout (CLTV) of the kth HTLC is set to height h_k for the `Offered` and `Received` outputs; prk_k is the corresponding preimage of the payment hash; $\Rightarrow$ denotes the punishment path; and 1 refers to an output.

this channel. These are carried by *hash time-locked contracts* (HTLCs). We discuss HTLCs fully in Sec. II-B. The protocol relies on two things: careful handling of revocation keys and careful handling of transaction signatures. Fig. 1 shows the channel lifecycle.

1) The Transactions: Although we abstract it away in our model, one should keep in mind the shape of the transaction in the Lightning Network. Fig. 2 gives the shape of the commitment transaction held by Alice in an Alice-Bob channel. This transaction notably enables unilateral closing. Outputs 3 and 4 enable the simple redistribution of the common fund between Alice and Bob, while Outputs 1 and 2 lock the funds of in-flight payments specific to the Lightning Network.

The figure also illustrates the punishment mechanism: unlike Bob's Output 3, Alice needs to wait for a relative timelock (CSV; common abbreviation for the `CheckSequenceVerify` opcode) before redeeming Output 4. That gives Bob the opportunity to redeem Output 4 using Alice's *revocation key* (rk_A). This revocation key is initially hidden from Bob. Alice only releases it as a proof that she will not use a given state. This mechanism enables declaring states as "old".

Fig. 2 also describes the local mechanism for in-flight payments. Output 1 (resp. 2) captures in-flight funds flowing towards Bob (resp. Alice). Besides the punishment mentioned above, these outputs are further governed by *payment hashes* and a timeout. To *fulfil* an in-flight transfer, the receiver (Bob in the case of Output 1) needs the preimage of the payment hash associated with that transfer. To prevent funds from being locked up, the output effectively reverts to a regular output once a block height is reached (e.g., h_i for Output 1). This absolute timeout is set with CLTV (common abbreviation for the `CheckLockTimeVerify` opcode). The real Lightning Network uses a two-stage construction to separate the CLTV and CSV conditions; Fig. 2 collapses it into a single output per HTLC.

Of this transaction, our model retains counterparts of Outputs 3 and 4 and of the revocation path, but not the in-flight Outputs 1 and 2: HTLCs are resolved in the routing layer, and the single on-chain spend of an HTLC output becomes the named assumption that each HTLC is resolved at most once (Sec. IV).

2) The Protocol: The lifecycle of a channel has five phases: opening, update, revocation, closing, and punishment, which correspond, left to right, to Fig. 1.

a) Opening: The opening phase aims to establish a first committed state and the common fund, in that order, to prevent griefing attacks. After an unmodelled negotiation of the inputs of the common fund, Alice and Bob each sample a random revocation key (rk^0_A and rk^0_B respectively) and exchange their hashes. Relying on SegWit [20], both parties can compute the TXID of the common fund and exchange the various signatures needed to spend it: the common fund is an on-chain output locked by both parties with a 2-of-2 multisig [19]. For instance, Bob signs the various outputs of Fig. 2 (Sec. V-A1 describes how our model abstracts these signatures). Then, the witnesses for the common fund are exchanged and the funding transaction is published. Once it is on chain, the channel is ready.

b) Update: The parties then pay each other off chain, by exchanging signed commitment transactions. Updating involves setting up a new state, then revoking the old one. The parties first commit to a new state via the same mechanism as in the opening phase: sampling revocation keys (rk^n_A and rk^n_B respectively for the nth state), exchanging their hashes, and exchanging the signatures. Only then do they revoke the old state (here the $(n-1)$th state) by disclosing their revocation keys to each other (e.g., A sends rk^{n-1}_A to B). If the steps were reversed, a party would have nothing to publish. The old state would be revoked, and the new one would not be signed. It could not close the channel if the counterparty stopped responding.

c) Revocation: A state is retired by handing its revocation key to the counterparty: Alice revokes state n by giving rk_A^n to Bob. The key is useless unless Alice later publishes that state, at which point it lets Bob take the whole channel balance. Revoking is therefore a proof of honesty, as publishing that old state could then result in punishment. A commitment that has not been revoked yet is a valid unilateral close: either party may publish it without penalty. Punishment is triggered only when a revoked commitment is published.

d) Closing: A channel can close in two ways. In the *cooperative* close, the parties agree on the final balance and sign one transaction together. This transaction sends the final balance directly to their wallets. Only one on-chain transaction is needed. There are no scripts and no delay. This is the cheapest outcome for both. In a *unilateral* close, either party can close: it publishes its current commitment alone. The publisher's own output (Output 4) is restricted by the CSV delay and by the revocation path. The party cannot spend its own output until the CSV delay ends. This delay gives the counterparty time to check whether the published state was revoked: if it was, the counterparty can take the output outright during that window. The counterparty's own output (Output 3) is spendable immediately. Therefore, unilateral closing disadvantages the publisher and favours the counterparty. This creates a punishment window, which is why an honest party closes unilaterally only when its counterparty has stopped responding.

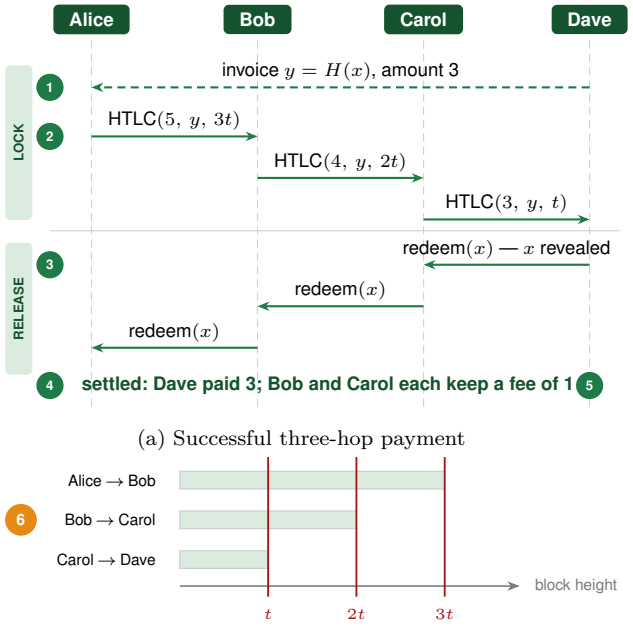
e) Punishment: If an old state is published on the blockchain by one party (i.e., a state whose revocation key is known to the counterparty), the counterparty can defend itself using the punishment paths shown in Fig. 2 and claim the entire channel balance. Punishment works as an *active* defence: the defender must act (i.e., notice the fraud, send the punishment transaction, and wait for it to be accepted) within the window given by the CSV. Failure to react makes the protocol trivially insecure [21].

B. Multi-hop HTLC Routing

When the sender and recipient do *not* share a direct channel, the payment is routed through intermediaries along the path as in Fig. 3: it is a *multi-hop* payment. Because intermediaries are untrusted, each hop relays the payment in an HTLC conditioned on a single shared secret, the *payment hash*. As a result, the payment is meant to either complete along the entire path or fail. The wormhole attack (end of this section and Sec. V-E) shows that colluding intermediaries can break this for the hops between them. Furthermore, to provide an incentive, the intermediaries are rewarded with a routing fee.

Example 1 (Path)**.** *We will use the running example where S sends money to R with the following path:*

$$S \to F_1 \to \cdots \to F_n \to R \tag{1}$$



(a) Successful three-hop payment

(b) Griefing

Fig. 3. In-flight payments on the route Alice→Bob→Carol→Dave. (a) Amounts decrease $5 \to 4 \to 3$; each one-unit drop is that intermediary's fee. Deadlines decrease $3t > 2t > t$, so a forwarder whose outgoing HTLC is claimed still has time to redeem its incoming one. (b) The HTLCs are locked as in (a), but Dave never releases x, so every HTLC runs to its deadline. Each window strictly contains the next. Capital is locked for the full window and the payment fails, but the nested deadlines $t < 2t < 3t$ mean that every honest party is refunded: griefing is a denial of service, not a loss of funds. Circled numbers index the lemma groups of Table II.

a) Locking: For this, the receiver (R in Ex. 1) generates a preimage x and sends an invoice to the sender (S) containing the hash $y = H(x)$ and the requested amount v, keeping x strictly secret. The sender adds an HTLC to its channel with F_1, locking funds v_0 on that channel conditional on y before the deadline d_0; after d_0, the payer may reclaim them. The condition to unlock the funds is to provide a preimage x' with $H(x') = y$ before the deadline (as in Fig. 2). This moves forward with each forwarder locking funds, the routing fee $v_{k-1} - v_k$ going to forwarder F_k. Each HTLC uses the same hash but a decreasing deadline and decreasing funds:

$$v_0 > v_1 > \cdots > v_n = v \tag{2}$$

$$d_0 > d_1 > \cdots > d_n \tag{3}$$

At this step, funds are still locked by the payment hash; they have not moved yet. Each HTLC becomes an additional conditional output in the two parties' current commitment transaction, as explained in Sec. II-A. The intermediaries F_i can safely lock funds, since an incoming HTLC has a larger amount and a later deadline. If the outgoing HTLC is claimed, F_i learns x and still has time to unlock the incoming funds, keeping the difference as its fee. Sec. V-A4 explains how our model abstracts amounts and deadlines.

b) Release Round: In the release round, the receiver reveals x, the preimage, to the last forwarder to claim the final HTLC. Since every HTLC has the same preimage, each intermediary claims on its incoming channel using the same x, each learning x when the next hop claims from it. To claim v_{n-1} from F_{n-1}, F_n provides the secret x for its incoming HTLC, and it keeps $v_{n-1} - v_n$ as a fee. The preimage thus propagates backward hop by hop until the sender's HTLC is settled. If any party in between fails to forward it and the timelock expires, the funds return to the party that locked them. Once the receiver has released x, if every intermediary claims its incoming HTLC in time, all HTLCs on the path are settled. An intermediary that fails to do so after its outgoing HTLC was redeemed loses the forwarded amount, while the parties downstream are paid. This is why forwarders need liveness (Remark 2).

Remark 1 (Claiming and Liveness)**.** *Claiming can happen by mutual negotiation or by unilateral closing. Notably, unilateral closing also publishes x, as it is leaked by the closing transactions redeeming the HTLC.*

Remark 2 (Decreasing Deadlines and Liveness)**.** *Consider F_i holding an incoming HTLC with deadline d_{i-1} and an outgoing HTLC with deadline d_i. If F_{i+1} unilaterally claims the outgoing HTLC at d_i, F_i learns x as late as d_i and is left the window (d_i, d_{i-1}) to settle its incoming HTLC. The gap between d_{i-1} and d_i is the CLTV delta, which is precisely the time reserved for F_i to recover the funds. This means that, in case of unilateral close, forwarders need liveness in order to claim their incoming HTLC within their claim window [21].*

Our model imposes this liveness as an explicit restriction, T3 (Sec. IV-B), and Sec. V-D shows that intermediary safety fails without it.

Finally, all HTLCs of a payment are locked by the same hash y, so two colluding intermediaries on the route can pass x around the hops between them: the payment completes while the bypassed hops are refunded and earn no fee. This is the wormhole attack [6]; Sec. V-E gives such a trace, checked by Tamarin, and Sec. VI proves that, for routes of any length, a hop of a settled payment whose hash is offered once can be bypassed only if some party's long-term key is compromised.

III. Symbolic Model and Notation

We model the protocol using Tamarin [16]. It is a symbolic tool for security-protocol analysis, which tracks a *system* represented as a multiset of facts, referred to as the state S. The state evolves following user-defined transition *rules* of the following shape:

$$[\,L\,] \text{---}[\,A\,]\longrightarrow [\,R\,] \tag{4}$$

with the premises L, the action facts A and the conclusions R.

A rule of shape (4) can be applied when L unifies with facts in S. It captures that L is rewritten as R in S, and the action facts A are appended to the trace. Tamarin features linear and persistent facts; the former are consumed upon rule application, while the latter (prefixed with `!`) can be reused arbitrarily many times. The tool also provides the built-in facts `In`, `Out` and `Fr`, capturing respectively receiving from and sending to a public (i.e., attacker-controlled) channel, and generating fresh and unique values (e.g., for nonces).

We further use the hashing, signing and natural-numbers built-ins. Hashing provides h, used for payment hashes $y = h(x)$ and for the commitment $h(s)$ to a revocation secret s. Signing provides `sign`, `verify` and `pk`, used for the channel handshake, the commitments and the invoices. Natural numbers provide the sort of amounts (variables prefixed with `%`) with addition, used for the per-hop constraint between incoming amount, outgoing amount and fee (Sec. V-A4).

An execution produces a trace of action facts. Security goals are then first-order formulas over these traces (with time points `#i`), enabling security queries over all traces (`all-traces` lemmas) and reachability queries, ensuring that a certain trace exists (`exists-trace` lemmas). A *restriction* is a formula of the same kind that Tamarin imposes on the model: only traces that satisfy every restriction are considered.

IV. Formal Modelling Methodology

We model the protocol as a single Tamarin theory, `lightning.spthy` (37 rules), with a single adversary. It contains both the payment-channel protocol (Sec. II-A) and multi-hop routing (Sec. II-B). One generic rule, `Forward_HTLC`, handles every forwarding hop, so routes of any length are covered. The routing layer does not abstract the channel layer away, but the coupling is deliberately thin: commitment transactions carry no HTLC outputs, and, besides the long-term keys, the layers interact only through three explicit interfaces.

a) Topology and Slot: `Lock_Funds_And_Open` mints the persistent facts `!ChannelConnect`, one per direction, which `Sender_Offer_HTLC`, `Forward_HTLC` and `Open_Timeout` require; HTLCs are therefore added only on opened channels (`HTLC_On_Opened_Channel`). Opening also mints one linear fact `Mutex(ptr)`: adding an HTLC consumes it, and redeeming or refunding that HTLC returns it. A channel thus holds at most one in-flight HTLC at a time and can be reused afterwards.

b) Resolution: Every rule that redeems or refunds an HTLC records the action fact `Old`, and each HTLC is resolved at most once:

```
1 restriction OneResolutionPerHTLC:
2     "All ptr y #i #j. Old(ptr, y) @ #i & Old(ptr, y) @ #j
3      ==> #i = #j"
```

This is the interface assumption between the layers. In the protocol, it holds because an HTLC output of the current commitment can be spent only once on chain; spending it from an older state requires publishing a

revoked commitment, which the counterparty can punish (`Publish_Revoked_A/B`, `Punish_A/B`). This argument is not mechanised, and we claim no composition theorem. All other abstractions are also explicit, named restrictions (Sec. IV-B).

A. Threat Model

Tamarin assumes the standard Dolev-Yao adversary model [10]: it controls the whole network; it can read, modify and inject messages of arbitrary complexity; it is, however, constrained to the term algebra under consideration. Our adversary keeps this full control. Public keys, channel proposals and acknowledgements, and invoices are sent over the network. The adversary also chooses the values that honest rules read from it: the balances of a channel update (`Update_State`) and, at each hop, the incoming amount, the fee and the outgoing amount (`Forward_HTLC`), subject only to the incoming amount being the outgoing amount plus the fee. HTLC offers and fulfilments between adjacent hops are linear facts (`Route`, `Fulfilled`), that is, authenticated, non-replayable messages between channel partners.

a) Key Compromise: Rule `Compromise_Ltk` reveals the long-term key of any party at any time (action fact `LtkCompromised`); the adversary can then sign as that party, e.g., forge its invoices: a payment for which the receiver created no invoice can then settle (`Forged_Invoice_Requires_Key_Compromise`). The preimage x is never sent on the network, and the adversary never learns it (`Preimage_Never_Known`). What compromised parties can do with it is given explicitly by rule `AttackerSkip`: two compromised parties that offered or forwarded HTLCs with the same payment hash may move the fulfilment of one party's outgoing HTLC to the other party's outgoing HTLC, relaying x around the parties between them: the wormhole attack [6] (Sec. V-E). Apart from their keys, `AttackerSkip` and the exemptions of Sec. IV-B, compromised parties follow the protocol rules.

b) Time: Time is abstract: the timeout of channel `ptr` is the event `TimedOut(ptr)` of rule `Open_Timeout`, which can happen at any point after the channel is opened, and at most once per channel (`OneTimeoutPerPtr`). The adversary therefore chooses when timeouts happen relative to honest actions.

c) Liveness: As mentioned before, the security of the Lightning Network relies on the liveness of the participants, whether to deter cheating behaviour (Sec. II-A2e) or to claim HTLCs (Remark 1). However, a standard Dolev-Yao adversary has full control over the network. It can notably keep a party from seeing the network (e.g., preventing it from noticing a fraud) or from sending a message (e.g., the defending transaction), leading to trivial attacks on the Lightning Network [21]. Our model therefore uses Tamarin restrictions to impose liveness on honest forwarders, as explained next.

B. Liveness Restrictions

To make the model relevant, we restrict the adversary's control over the order of events to allow for liveness. We also rule out pathological honest behaviour. Concretely, we impose the following three restrictions, which constrain the routing layer (shown without the flags `NO_RBT`, `NO_OTF`, `NO_IMC` that remove them for the necessity runs).

```
restriction RedeemBeforeTimeout:
    "All ptr y #i #j.
        Redeemed(ptr, y) @ #i & TimedOut(ptr) @ #j ==> #i < #j"
restriction OutgoingTimesOutFirst:
    "All P Prev Next ptrIn ptrOut y #f #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrIn) @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> Ex #m. TimedOut(ptrOut) @ #m & #m < #k"
restriction IntermediaryMustClaim:
    "All P Prev Next ptrIn ptrOut y #f #i #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & Redeemed(ptrOut, y) @ #i
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> Ex #l. Redeemed(ptrIn, y) @ #l"
```

Restriction T1 (`RedeemBeforeTimeout`). *If an HTLC is redeemed and its channel times out, the redemption comes first.* This is not a liveness assumption but a well-formedness condition on timeouts, and it applies to every HTLC.

Restriction T2 (`OutgoingTimesOutFirst`). *If the incoming HTLC of a forwarding hop times out and the forwarder was not compromised before, then its outgoing HTLC timed out earlier.* This is the decreasing-deadline (CLTV-delta) invariant of Remark 2.

Restriction T3 (`IntermediaryMustClaim`). *If a forwarder's outgoing HTLC is redeemed and its incoming HTLC times out, and the forwarder was not compromised before that timeout, then its incoming HTLC is redeemed too.* This is the claiming liveness of Remarks 1 and 2, on which intermediary safety rests. It only constrains traces in which the incoming HTLC times out, the only ones in which the upstream party can be refunded (`Refund_Requires_Timeout`); by T1, the claim then precedes the timeout.

T2 and T3 do not constrain forwarders compromised before their incoming timeout. The order of a hop's two timeouts is not assumed separately: Tamarin proves it from T2 and `OneTimeoutPerPtr`.

```
lemma ForwardTimeGap [reuse]:
    "All P Prev Next ptrIn ptrOut y #f #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrOut) @ #j
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> #j < #k"
```

As restrictions, T1 to T3 remove every trace that violates them, including traces in which an uncompromised forwarder could not have acted in time; our results say nothing about such traces (Sec. V-D). So, in our model, the liveness of punishment (Sec. II-A2e) is not encoded: no restriction mentions cheating or punishment.

The six remaining restrictions are structural. `Equality` and `Inequality` turn the action facts `Eq` and `Neq` into

constraints, which encodes checks such as signature verification; `OneChannelPerPtr` and `OneTimeoutPerPtr` let a channel open and time out at most once; `OneHTLCPerHashPerChannel` lets a payment hash be added at most once per channel; and `OneResolutionPerHTLC` is the interface assumption above.

Sec. V-D removes one liveness restriction at a time. T3 is necessary for intermediary safety: without it, TAMARIN finds a trace with no key compromise in which an intermediary's outgoing HTLC is redeemed while its incoming one is refunded. T1 is not needed by any of the five liveness lemmas checked there, and T2 is not needed for intermediary safety. The bypass theorem for routes of any length and a single offer per payment hash (Sec. VI-B) is proved without T1, T2 and T3; all other results of the main theory are proved under all restrictions.

C. Structural Limitations

The Lightning Network is a complex protocol. Verifying it as a single monolithic block quickly hits scalability problems with verification tools [8], and TAMARIN is no exception. This is not an assumption but an observation. When different built-ins are combined in TAMARIN, the search space explodes: with signatures, an equational theory, natural-number induction and an unbounded clock, TAMARIN exhausts memory; without the unbounded clock, the remaining ones are tractable together. We therefore keep the block clock out of the main theory, which represents timeouts as untimed events ordered by the restrictions above. The block clock and the CLTV arithmetic are analysed in small auxiliary models that use hashing and natural numbers only.

Our artifact thus consists of one main theory and five small auxiliary models, all in `lightning_aux.spthy`, each selected by a preprocessor flag: the PTLC countermeasure (`-D=PTLC`) and its HTLC baseline (`-D=HTLC`), a block-clock model of CLTV deadlines (`-D=CLOCK`), CLTV-delta arithmetic (`-D=CLTV`) and the early-timeout race (`-D=TIMEOUT`). Each is self-contained and is not composed with the main theory (Table I). Inside the main theory, HTLC state is deliberately not folded into the commitment transactions: `StateA` and `StateB` carry balances and revocation data but no HTLC outputs. This keeps the channel state independent of the payments routed over it. The price is `OneResolutionPerHTLC`, imposed rather than derived from the channel mechanics; a mechanised discharge, by composition or by refinement as in [8], is the natural next step.

Five aspects are deliberate non-goals, each of which buys generality or tractability. *Onion privacy:* each forwarder may pick any channel partner as its next hop and nothing is encrypted in layers, so the results cover every route but claim no privacy. *On-chain fees:* publishing a transaction is free, which separates the protocol logic from fee economics. *Fee policy:* the adversary chooses every routing fee, so the results hold for any policy. *Wallet capacity:* HTLC amounts are not checked against the balances, so the results hold for all amounts. *Per-HTLC CSV stage:* an HTLC is redeemed or refunded in one step, without a second-stage transaction.

V. Verification of the Model

This section describes how the model is built (Sec. V-A) and what we verified (Sec. V-B). Listings are verbatim excerpts of `lightning.spthy`; `...` marks elisions.

A. The Core Model

As stated in Sec. IV, the main theory contains both layers, as two groups of rules that share only the long-term keys and the channel topology.

1) Payment Channels: A channel is identified by a fresh pointer `ptr` to its founding transaction. Our model tracks two mirror facts about this channel, `StateA` and `StateB`, one held by each party, each carrying the two balances (`fA`, `fB`), a state index `n`, and that party's own revocation secret together with the corresponding commitment. The commitment is stored without its signature; signatures are checked when a state is installed (`Both_States_Committed` for the first, `Revoke_Old_Secret` for every later one).

The heart of the model is the *update dance*: commit a new state first, then revoke the old one. `Update_State` samples the next state index and both fresh revocation secrets, reads the two current mirrors and the new balances from the environment, and produces two pending mirrors without destroying anything: each pending mirror keeps the old state next to the new, signed one.

```
rule Update_State:
    let newCommitA = <'commit', ptr, $A, $B, ~n_new, h(~sAnew)>
        newCommitB = <'commit', ptr, $B, $A, ~n_new, h(~sBnew)>
        sigAnew    = sign(newCommitA, ltkA)
        sigBnew    = sign(newCommitB, ltkB)
    in
    [ ...
      StateA($A, $B, ptr, n_old, sAold, oldCommitA, fA, fB),
      StateB($A, $B, ptr, n_old, sBold, oldCommitB, fA, fB),
      In(<fA_new, fB_new>) ]
    --[ NewStateBuilt($A, $B, ptr, ~n_new, fA_new, fB_new) ]->
    [ PendingA($A, $B, ptr, n_old, sAold, oldCommitA,
        ~n_new, ~sAnew, newCommitA, sigAnew, fA_new, fB_new),
      PendingB(...) ]
```

Only afterwards does `Revoke_Old_Secret` supersede the old state: it consumes the two pending mirrors, installs them as the new state, and, crucially, records the old revocation secrets as persistent facts.

```
rule Revoke_Old_Secret:
    [ PendingA($A, $B, ptr, n_old, sAold, oldCommitA,
        n_new, sAnew, newCommitA, sigAnew, fA_new, fB_new),
      PendingB($A, $B, ptr, n_old, sBold, oldCommitB,
        n_new, sBnew, newCommitB, sigBnew, fA_new, fB_new),
      !Pk($A, pkA), !Pk($B, pkB) ]
    --[ StateUpdate($A, $B, ptr, n_new),
        Revoke($A, n_old, sAold),
        Revoke($B, n_old, sBold),
        Eq(verify(sigAnew, newCommitA, pkA), true),
        Eq(verify(sigBnew, newCommitB, pkB), true) ]->
    [ StateA($A, $B, ptr, n_new, sAnew, newCommitA, fA_new,
      ↪ fB_new),
      StateB($A, $B, ptr, n_new, sBnew, newCommitB, fA_new,
      ↪ fB_new),
      !RevokedSecret($A, n_old, sAold, oldCommitA),
      !RevokedSecret($B, n_old, sBold, oldCommitB),
      RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA),
```

```
      RevokedCommitmentUnspent(ptr, $B, $A, n_old, oldCommitB) ]
```

Two details deserve emphasis. First, the signature checks of Sec. II-A become the two `Eq` action facts; a TAMARIN rule cannot fail, so restriction `Equality` turns these action facts into hard prerequisites. Second, the old revocation secret becomes persistent knowledge, while the old commitment's spend becomes a linear one-shot token. This keeps the secret available for later punishment while guaranteeing that a given revoked commitment is spent at most once, following the single-spend discipline of Sec. V-B. The order (commit, then revoke) is what protects a party that aborts mid-update: the old state is never revoked until a fully signed new one exists.

The remaining phases reuse the same facts, so we give only the closing path here and trust the pattern carries over. `Lock_Funds_And_Open` mints the two mirrors together with the persistent facts `!ValidOnChain(ptr)`, `!ChannelRecord(ptr,$A,$B)` and `!ChannelConnect` (one per direction), and the per-channel slot `Mutex(ptr)`, which the routing rules take and give back (Sec. V-A2). A unilateral close spends the current mirror: `Publish_Current_A` turns `StateA` into an `OnChainDelayed` and an `OnChainInstant` output,

```
rule Publish_Current_A:
    [ StateA($A, $B, ptr, n, sA, commitA, fA, fB),
      ↪ !ValidOnChain(ptr) ]
    --[ Publish($A, n, commitA),
        ...
        OnChainInstantProduce($B, n) ]->
    [ OnChainDelayed($A, n, commitA),
      OnChainInstant($B, n, commitA) ]
```

and `Claim_Funds_A` later releases the delayed output to its owner:

```
rule Claim_Funds_A:
    [ OnChainDelayed($A, n, txContent) ]
    --[ ClaimFunds($A, n), CSVPassed($A, n), SettledTx($A,
      ↪ txContent) ]->
    [ FundsTo($A, n) ]
```

Notably, the `CSVPassed` action lets us capture that time has passed. A cooperative close (`Honest_Close`) signs one shared settlement transaction instead, and punishment (`Publish_Revoked_A` consuming `RevokedCommitmentUnspent`, then `Punish_A`) transfers the whole channel to the honest party.

2) Routing: The routing layer of Sec. II-B is a chain of HTLC offers in the forward direction and of fulfilments in the backward one, and our model gives each direction its own fact. Forward obligations travel as linear facts `Route($Prev,$Me,ptrIn,y)`, and every added HTLC is also recorded in a persistent fact `!RouteInfo`. Once an HTLC is redeemed, the preimage travels back as `Fulfilled(ptr,x)`. The slot `Mutex(ptr)` acts as a per-channel lock: it is taken when an HTLC is added and given back when that HTLC resolves (Sec. V-B).

The core of the model is the generic forwarding rule, one rule for any number of hops.

```
rule Forward_HTLC:
    [ Route($Prev, $Me, ptrIn, y),
      !ChannelConnect(ptrOut, $Me, $Next),
      Mutex(ptrOut),
      In(%vIn), In(%fee), In(%vOut) ]
    --[ HTLCForwarded($Me, $Prev, $Next, ptrIn, ptrOut, y),
        HTLCAdded(ptrOut, y),
        FeeCharged($Me, y, %fee),
        Eq(%vIn,  %vOut %+ %fee),
        Neq(ptrIn, ptrOut), Neq($Prev, $Me), Neq($Me, $Next) ]->
    [ IntermediaryPending($Me, $Prev, $Next, ptrIn, ptrOut, y,
      ↪ %vIn, %vOut, %fee),
      Route($Me, $Next, ptrOut, y),
      !RouteInfo($Me, $Next, ptrOut, y) ]
```

`Forward_HTLC` advances the payment one hop: it consumes the incoming `Route` fact, produces the outgoing one, takes the slot `Mutex(ptrOut)` of the outgoing channel, and stores in `IntermediaryPending` everything the hop needs to settle or to be refunded later. `$Next` is any party with which `$Me` shares a channel, so routes are built hop by hop and have no fixed length; this single rule serves every forwarding hop of every route. The amounts and the fee are chosen by the environment, constrained only by the `Eq` action fact (Sec. V-A4). The action `HTLCForwarded` identifies the hop in our lemmas, `HTLCAdded` feeds restriction `OneHTLCPerHashPerChannel`, and `FeeCharged` records the fee the hop expects. The `Neq` facts make the forwarder distinct from both of its neighbours and the two channels distinct from each other.

The settle round runs in the other direction on separate facts. `Receiver_Create_Invoice` signs a tagged invoice that names the receiver, the sender and the payment hash $y = h(x)$ of a fresh preimage x, sends it with its signature, and keeps x private in `ReceiverInv`. `Sender_Offer_HTLC` checks that signature before offering the first HTLC. `Receiver_Fulfill_HTLC` consumes the last `Route` fact, checks that x hashes to y, emits `Redeemed(ptrIn,y)` and passes the preimage upstream as `Fulfilled(ptrIn,x)`. The preimage is never sent over the network, and the adversary never learns it (`Preimage_Never_Known`). `Intermediary_Settle` then consumes `IntermediaryPending` and `Fulfilled(ptrOut,x)`, produces `Fulfilled(ptrIn,x)` one hop upstream, and records `FeeEarned` when the intermediary collects its fee. Both rules give the slot `Mutex(ptrIn)` back. `Sender_Settle` finally consumes `SenderPending` and `Fulfilled(ptrSF,x)` and closes the payment with `SenderSettled`.

Finally, the model lets a payment fail without punishment. `Open_Timeout` marks a channel as timed out (an untimed event, Sec. IV-A), which makes the pending HTLC on that channel refundable: `Sender_Timeout_Refund` and `Intermediary_Timeout_Refund` consume the corresponding pending state to return funds (recording `Refunded`) and give the slot `Mutex` back. The interplay of claiming and refunding is pinned down by the three liveness restrictions of Sec. IV-B: T1 forbids redeeming after the timeout, T2 encodes the decreasing deadlines of Remark 2, and T3 the claiming liveness of Remarks 1 and 2.

3) The Wormhole Rule: Besides the Dolev-Yao network and key compromise (`Compromise_Ltk`), the adversary has one routing rule, used for the wormhole:

```
rule AttackerSkip:
```

```
    [ Fulfilled(ptrExit, x),
      !RouteInfo(ExitAtt, Next, ptrExit, y)[-,no_precomp],
      !RouteInfo(EntryAtt, Skipped, ptrSkip, y)[-,no_precomp],
      !Compromised(ExitAtt),
      !Compromised(EntryAtt) ]
    --[ FulfilledProduced(ptrSkip, x),
        Skipped(ptrSkip, x),
        Eq(h(x), y) ]->
    [ Fulfilled(ptrSkip, x) ]
```

If two compromised parties, `ExitAtt` and `EntryAtt`, have each added an HTLC for the same payment hash y, on channels `ptrExit` and `ptrSkip`, then `AttackerSkip` takes the preimage released on `ptrExit` and delivers it on `ptrSkip`. `EntryAtt` can then settle upstream, while the hops between `ptrSkip` and `ptrExit`, starting with `Skipped`, never see their outgoing HTLC fulfilled and can at best be refunded. The rule emits `Skipped(ptrSkip,x)` but neither `Redeemed` nor `FeeEarned`. It yields the wormhole trace of Sec. V-E (`Wormhole_Fee_Theft_Reachable`), and the hop-bypass theorems (Secs. V-E and VI) show that, if the sender settles, a forwarder at positions 1 to 6 of the route, or at any position of a single-offer payment, is bypassed only if some key was compromised.

4) Amounts and Fees: Amounts and fees are modelled per hop only. `Forward_HTLC` reads the incoming amount, the fee and the outgoing amount from the network as natural-number terms and requires the incoming amount to equal the outgoing amount plus the fee, so this balance holds at every hop by construction. The three values are kept in `IntermediaryPending`. `Sender_Offer_HTLC`, the invoice and the `Route` facts carry no amount, so amounts are not chained along the path, and the model makes no statement about end-to-end value, such as the amount the receiver obtains or the total fee the sender pays. We keep per-hop fees because the wormhole of Sec. V-E is a fee theft. The fee is a single per-hop quantity rather than Lightning's `base_fee + fee_rate` formula, and we prove nothing about fee policy. Deadlines are not numbers either: T2 captures only their order (Sec. IV-B), and the block clock and the CLTV arithmetic are checked in the auxiliary models of Table I.

Fees are charged and earned by two separate events. `FeeCharged` is emitted when a forwarder locks its outgoing HTLC; `FeeEarned` is emitted only when it later claims its incoming one. In the wormhole trace, the bypassed honest hop emits `FeeCharged` but never `FeeEarned`, although the sender settles and the receiver is paid (`Wormhole_Fee_Theft_Reachable`, Sec. V-E). Conversely, a hop whose key is never compromised earns a fee only after it has forwarded the payment and its outgoing HTLC has been redeemed (`Fee_Only_On_Successful_Forward`).

We deliberately leave out three things (see also Sec. IV-C). First, channel capacity: `Forward_HTLC` has no `Wallet` premise and HTLCs do not change the balances in `StateA` and `StateB`, so a forward never fails for lack of funds. Second, concurrent HTLCs on one channel: a channel carries at most one pending HTLC at a time (Sec. V-B); later payments can reuse it, each with a payment hash not used on that channel before (restriction `OneHTLCPerHashPerChannel`). Third, onion routing: the sender chooses only the first hop, each forwarder chooses its next hop among its own channels, and route information is kept in local state rather than sent in layered encryption. We therefore make no claims about hiding who pays whom, and the wormhole appears purely as fee theft, as also characterised in [6]. Similarly, `StateA` and `StateB` carry a state index and a revocation secret, not the output structure of Fig. 2, and the two-stage HTLC claim becomes a single transition. The second stage adds a CSV delay and a revocation path; for the channel balance, these are enforced by `Publish_Current_A/B`, `Claim_Funds_A/B` and the punishment rules.

B. Verification Results

Table I lists every verification run of our artifact, one per `Makefile` target; `make all` runs them in order. All results were obtained with Tamarin 1.10 (which reports the Maude version used, 3.2, as unsupported) and reproduced on a laptop with a 6 GB heap limit (`MEM=6G`).

`lightning.spthy` is our main model. It contains the channel layer, multi-hop routing with the single generic rule `Forward_HTLC` and the wormhole rule `AttackerSkip` (Sec. V-A), all under one Dolev-Yao adversary that can compromise long-term keys. Its default mode (`make verify`) proves the safety lemmas about channels and payments (Secs. V-C to V-E, Appendix A) and that the forwarders at positions 1 to 6 of a route cannot be bypassed unless some key was compromised (Sec. VI-A). Its exists-trace lemmas show that the model is not vacuous (channel updates, cooperative closes, refunds and honest multi-hop payments are reachable), that the assumptions of our safety lemmas matter (a payment without a matching invoice can settle once the receiver's key is compromised; a compromised intermediary can lose), and that the wormhole is reachable. `make general` drops the three liveness restrictions and proves the bypass theorem for routes of any length, for payments with a single offer per payment hash (Sec. VI-B); since removing restrictions only adds traces, this all-traces result also holds in the default mode. Of the five auxiliary models (Sec. IV-C), the PTLC countermeasure and its HTLC baseline are discussed in Sec. VI-E; the two timing models make explicit the deadlines that the main model abstracts into untimed timeout events; and the early-timeout race is the trace pattern that `Timeout_Race_Blocked` excludes in the main model for uncompromised forwarders. `make necessity` removes one liveness restriction at a time (Sec. V-D).

a) The Linear Single-Spend Discipline: We enforce the no-double-spend rule using linear facts. As mentioned in Sec. III, a persistent fact `!F()` is checked by a rule but not consumed, whereas a linear fact `F()` is removed from the state when the rule fires. In our model of the Lightning Network, the revocation secret is persistent knowledge (`!RevokedSecret`), and the revoked commitment output

TABLE I

VERIFICATION RUNS OF THE ARTIFACT (TAMARIN 1.10), ONE ROW PER `MAKEFILE` TARGET; `MAKE ALL` RUNS THEM IN THIS ORDER. A: ALL-TRACES LEMMAS; E: EXISTS-TRACE LEMMAS, ALL FOUND AUTOMATICALLY EXCEPT `WORMHOLE_FEE_THEFT_REACHABLE`, WHICH IS REPLAYED FROM A SAVED 116-STEP PROOF; VERIFIED: LEMMAS VERIFIED OUT OF THOSE RUN. BUILT-INS: HASHING, SIGNING AND NATURAL-NUMBERS IN `LIGHTNING.SPTHY`; HASHING AND NATURAL-NUMBERS IN `LIGHTNING_AUX.SPTHY`.

Target	File and flags	Content	A	E	Verified
`make verify`	`lightning.spthy`	Main model, all restrictions: 32 safety lemmas; 13 hop-bypass lemmas (six theorems for positions 1 to 6, seven helpers); 10 exists-trace lemmas	45	10	55/55
`make general`	`lightning.spthy -D=GENERAL -D=NO_IMC -D=NO_OTF -D=NO_RBT -D=NECESSITY`	Liveness restrictions T1, T2, T3 removed: `Any_Hop_Skip_Requires_Compromise` (any route length, single offer per payment hash; 7 steps), its 7 helpers, and 16 lemmas of `make verify` re-proved in this mode	24	0	24/24
`make aux`	`lightning_aux.spthy -D=PTLC`	PTLC countermeasure on the fixed route S, H1, H2, H3, R: `No_Wormhole_Around_Honest_Hop` (25 steps); 2 control, 2 sanity traces	1	4	5/5
	`lightning_aux.spthy -D=HTLC`	HTLC baseline on the same route: `Wormhole_Around_Honest_Hop` is reachable (12 steps); 2 sanity traces	0	3	3/3
	`lightning_aux.spthy -D=CLOCK`	Block clock: channel and HTLC lifecycle, claim windows, preimage timing, exclusive outcomes; 3 reachability traces	6	3	9/9
	`lightning_aux.spthy -D=CLTV`	CLTV-delta arithmetic: positive gap, non-empty claim window, staggered path	3	0	3/3
	`lightning_aux.spthy -D=TIMEOUT`	Four rules, no timeout ordering: the early-timeout race is reachable (`Early_Timeout_Race`)	0	1	1/1
`make necessity`	`lightning.spthy -D=NECESSITY` and one of `-D=NO_IMC`, `-D=NO_OTF`, `-D=NO_RBT`	One liveness restriction (T3, T2 or T1) removed per run; the 5 liveness lemmas re-checked, 600 s limit each (Table III); counterexample `Necessity_IMC_Honest_Intermediary_Loses` (`-D=NO_IMC`, 28 steps)	5×3	1	10/15 checks (5 without result); 1/1
Total	verify, general, aux	16 of the 24 proofs of `make general` re-prove lemmas of `make verify` in the liveness-free mode	79	21	100/100 (plus necessity runs)

is a linear one-shot token (`RevokedCommitmentUnspent`), consumed when that state is published.

Initially, the punishment rule only needed one thing to trigger: the persistent revocation secret alone. Our first `Publish_Revoked_A` had all the premises of the rule below except the last one, so every premise was persistent. Since punishment was gated only by `!RevokedSecret`, the rule could fire again and again for the same revoked state: if a party cheats, the honest party can take the channel funds many times. Lemmas like `No_Punishment_Without_Cheating` still pass, because a cheat is still required. What they miss is that punishment should happen only once. To fix this issue, we split the revoked state into two separate facts: `Revoke_Old_Secret` (Sec. V-A1) still stores the secret as the persistent `!RevokedSecret`, and it also creates the linear token `RevokedCommitmentUnspent`, for both parties. `Publish_Revoked_A` must consume that token, so the same revoked commitment can be punished only once (`Publish_Revoked_B` is symmetric):

```
rule Publish_Revoked_A:
    [ !ChannelRecord(ptr, $A, $B),
      !ValidOnChain(ptr),
      !RevokedSecret($A, n_old, sAold, oldCommitA),
      RevokedCommitmentUnspent(ptr, $A, $B, n_old, oldCommitA) ]
    --[ PublishRevoked($A, n_old, oldCommitA),
        Cheat($A, n_old) ]->
    [ OnChainCheat($A, $B, n_old, oldCommitA),
      RevealedSecret($A, n_old, sAold, oldCommitA) ]
```

Persistent knowledge of the revocation secret is retained via `!RevokedSecret`. Only spending the funds is linear.

We found the same critical flaw in our initial model of the routing layer. With the persistent `!ChannelConnect` alone, one channel could add unboundedly many HTLCs on the same output. We fixed it with a linear slot `Mutex(ptr)`. `Lock_Funds_And_Open` produces it once, when the channel opens; `Sender_Offer_HTLC` and `Forward_HTLC` consume it when they add an HTLC on that channel; `Receiver_Fulfill_HTLC`, `Intermediary_Settle`, `Sender_Timeout_Refund` and `Intermediary_Timeout_Refund` give it back when that HTLC is redeemed or refunded. HTLCs are thus added to a channel one at a time, and the channel can be reused once its HTLC has resolved.

```
rule Lock_Funds_And_Open:
    [ ReadyToLock($A, $B, ptr, sA0, sB0, commitA0, commitB0),
      Wallet($A, ~fA), Wallet($B, ~fB) ]
    --[ ChannelOpen($A, $B, ptr), FundsLocked($A, $B, ptr),
        NewStateBuilt($A, $B, ptr, '0', ~fA, ~fB) ]->
    [ StateA(...), StateB(...),
      !ValidOnChain(ptr),
      !ChannelRecord(ptr, $A, $B),
      !ChannelConnect(ptr, $A, $B),
      !ChannelConnect(ptr, $B, $A),
      Mutex(ptr) ]
```

Adding a linear premise can remove some exists-trace witnesses. So we re-ran the whole of `lightning.spthy` after both fixes: all 55 lemmas of `make verify` are verified, including the ten exists-trace lemmas (Table I).

C. Protocol Properties

We state the guarantees of `lightning.spthy` as lemmas. Each is named after the TAMARIN property that establishes it. Adversarial capability is explicitly modelled as long-term key compromise (`Compromise_Ltk`). In addition, two compromised parties on the same payment can relay a preimage to each other around the parties between them (rule `AttackerSkip`, Sec. V-A3). We apply honesty assumptions explicitly, in each lemma, not globally; the exceptions are the restrictions of Sec. IV-B, which hold in every trace. Unless stated otherwise, every lemma of this section is proved in the default mode of `lightning.spthy` (`make verify`), under all nine restrictions, including T1 to T3.

Table II sorts all lemmas into groups, one per stage of a payment that they constrain. Each circled number in Figs. 3 and 4 matches the group that constrains that stage. We present the central lemmas below; Appendix A gives every statement.

The first guarantee is atomicity: on a route with two forwarders and no key compromise, once the receiver has redeemed its HTLC, the sender can no longer be refunded.

Lemma 1 (Payment Atomicity)**.** *For any two consecutive forwarders F_1 and F_2 on a payment with hash y (the incoming HTLC of F_2 is the outgoing HTLC of F_1): if the outgoing HTLC of F_2 on y is redeemed and no party's key was ever compromised, then the incoming HTLC of F_1 on y is never refunded, by anyone.*

```
lemma Payment_Atomicity_Under_Liveness:
  "All F1 F2 Prev1 Next1 Prev2 Next2 ptrSF1 ptrF1F2 ptrF2R y #i
     ↪ #j #k.
      HTLCForwarded(F1, Prev1, Next1, ptrSF1, ptrF1F2, y) @ #i
    & HTLCForwarded(F2, Prev2, Next2, ptrF1F2, ptrF2R, y) @ #j
    & Redeemed(ptrF2R, y) @ #k
    & (not Ex P #l. LtkCompromised(P) @ #l)
   ==> not (Ex Up #m. Refunded(Up, ptrSF1, y) @ #m)"
```

Since `Forward_HTLC` is one generic rule, F_1 and F_2 may sit anywhere on a route of any length; on a three-hop route, `ptrSF1` is the sender's HTLC and `ptrF2R` the receiver's. `Sanity_Atomic_Payment_Completes` shows that the premises are satisfiable.

The second guarantee protects the intermediaries, who lock their own funds for a payment that is not theirs.

Lemma 2 (Intermediary Safety)**.** *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then that forwarder's long-term key was compromised.*

```
lemma Intermediary_Never_Loses_Under_Liveness:
  "All P Prev Next Up ptrIn ptrOut y #i #j #k.
      HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
    & Redeemed(ptrOut, y) @ #j
    & Refunded(Up, ptrIn, y) @ #k
   ==> Ex #l. LtkCompromised(P) @ #l"
```

Lemma 2 does not say when the key was compromised: a compromise after the refund also satisfies it. The strong form closes this gap.

Lemma 3 (Intermediary Safety, Strong Form)**.** *If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then that forwarder's key was compromised before its incoming HTLC timed out.*

```
lemma Intermediary_Never_Loses_Strong:
  "All P Prev Next Up ptrIn ptrOut y #i #j #k.
      HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
    & Redeemed(ptrOut, y) @ #j
    & Refunded(Up, ptrIn, y) @ #k
   ==> Ex #l #t. LtkCompromised(P) @ #l & TimedOut(ptrIn) @ #t &
     ↪ #l < #t"
```

Lemma 3 matches the honesty condition of T2 and T3, which bind a forwarder only if it is not compromised before its incoming HTLC times out. Both lemmas quantify over every forward, so they hold for every hop of a route of any length, in every trace that satisfies T1, T2 and T3.

Routing causality says that nothing is forwarded out of thin air.

Lemma 4 (Routing Causality)**.** *Every forward on hash y is preceded by the HTLC it forwards: either the sender's offer on the forwarder's incoming channel, or the predecessor's forward that locked that channel's HTLC.*

```
lemma Forward_Requires_Incoming [use_induction]:
  "All Me Prev Next ptrIn ptrOut y #i.
      HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y) @ #i
   ==> (Ex S #j. HTLCOffered(S, Me, ptrIn, y) @ #j & #j < #i)
     | (Ex P0 pin0 #j. HTLCForwarded(Prev, P0, Me, pin0, ptrIn,
     ↪ y) @ #j & #j < #i)"
```

Because `lightning.spthy` has a single generic `Forward_HTLC` rule, this one lemma covers every hop of a route of any length, where a model with a fixed path needs one lemma per position (and an honest variant of each). It needs no honesty premise, because a forward consumes the linear `Route` fact, which only `Sender_Offer_HTLC` and `Forward_HTLC` produce. On the release side, an HTLC is redeemed only by the receiver's fulfilment or by a forwarder settling that hop (`Redeemed_Source`), and every preimage that a forwarder or the sender takes from a channel in order to settle comes from an earlier redemption of the HTLC on that channel or from an earlier wormhole skip (`Fulfilled_Source`, Sec. V-E).

Finally, a forwarder is paid for its service only if the service succeeded.

Lemma 5 (Fees Only on Success)**.** *A forwarder whose key is never compromised earns the fee of a hop only after it has forwarded on that hop and its outgoing HTLC has been redeemed.*

```
lemma Fee_Only_On_Successful_Forward
     ↪ [heuristic={route_info_first}]:
  "All Me ptrIn ptrOut y %fee #i.
      FeeEarned(Me, ptrIn, ptrOut, y, %fee) @ #i
    & not (Ex #c. LtkCompromised(Me) @ #c)
   ==> (Ex #j. Redeemed(ptrOut, y) @ #j & #j < #i)
     & (Ex Prev Next #f. HTLCForwarded(Me, Prev, Next, ptrIn,
     ↪ ptrOut, y) @ #f & #f < #i)"
```

Per hop, the incoming amount is the outgoing amount plus the fee by construction (the `Eq` action of `Forward_HTLC`, Sec. V-A4). Amounts are not chained along the route,

TABLE II

LEMMA GROUPS OF `lightning.spthy`; THE CIRCLED NUMBERS IN FIGS. 3 AND 4 REFER TO GROUPS 1 TO 7. A: ALL-TRACES; E: EXISTS-TRACE. UNMARKED ROWS: THE 55 LEMMAS OF `make verify`, ALL VERIFIED (32 ALL-TRACES SAFETY LEMMAS, THE 13 HOP-BYPASS LEMMAS OF GROUP 9, 9 EXISTS-TRACE LEMMAS FOUND AUTOMATICALLY, AND `Wormhole_Fee_Theft_Reachable`, REPLAYED FROM A SAVED 116-STEP PROOF). †: PROVED IN `make general` (LIVENESS RESTRICTIONS REMOVED; SEC. VI). ‡: EXISTS-TRACE, CHECKED WITH T3 REMOVED (TABLE III).

#	Group	Lemmas	Kind
0	Channel layer, keys	`state_update`, `delayed_funds`, `instant_funds`, `settlement_is_traceable`, `No_Punishment_Without_Cheating`, `Funds_Locked_Before_Update`, `Update_Requires_Negotiation`, `Ltk_Known_Implies_Compromised`	A
	non-vacuity	`Protocol_execution`, `Cooperative_Close_Execution`	E
1	Invoice & preimage	`Invoice_Authenticates_Settlement`, `Preimage_Secret_Until_Released`, `Preimage_Never_Known`, `Invoice_Has_Secret_Preimage`, `Invoice_Released_Once`	A
	reachable	`Forged_Invoice_Requires_Key_Compromise`	E
2	Lock causality	`Forward_Requires_Incoming`, `HTLC_On_Opened_Channel`, `Offer_Requires_Invoice`	A
3	Release causality	`Claim_Requires_Release`, `Settle_Requires_Receiver_Release`, `Redeem_Requires_Receiver_Release`, `Fulfilled_Requires_Release`, `Fulfilled_Source`	A
4	Atomicity	`Payment_Atomicity_Under_Liveness`, `Settle_Excludes_Sender_Refund`, `OneOutcomePerHTLC`	A
	non-vacuity	`Multihop_Routing_Reachable`, `Sanity_Honest_Payment_Completes`, `Sanity_Atomic_Payment_Completes`, `Distinct_Parties_Configuration`	E
5	Fees	`Fee_Only_On_Successful_Forward` (per hop, incoming amount = outgoing amount + fee by construction)	A
6	Refund / griefing	`Refund_Requires_Timeout`, `Loss_Requires_Inaction`	A
	reachable	`Refund_Possible`	E
7	Wormhole	`Skipped_Requires_Compromise`	A
	reachable	`Wormhole_Fee_Theft_Reachable` (saved 116-step proof, replayed)	E
8	Intermediary safety, timing	`Intermediary_Never_Loses_Under_Liveness`, `Intermediary_Never_Loses_Strong`, `Timeout_Race_Blocked`, `ForwardTimeGap`	A
	non-vacuity	`Sanity_Compromised_Intermediary_Can_Lose`	E
	necessity ‡	`Necessity_IMC_Honest_Intermediary_Loses`	E
9	Hop bypass	`First_Hop_Skip_Requires_Compromise`, `Second_Hop_Skip_Requires_Compromise`, `Hop3_Skip_Requires_Compromise`, `Hop4_Skip_Requires_Compromise`, `Hop5_Skip_Requires_Compromise`, `Hop6_Skip_Requires_Compromise`	A
	helpers	`Forward_Unique_Per_Incoming`, `SettledHop_From_Forward`, `SettledHop_Uses_Outgoing`, `No_Settle_And_Refund_Same_Hop`, `No_Fulfill_On_Forwarded_Incoming`, `Redeemed_Source`, `SenderSettled_Uses_Fulfilled`	A
	any length †	`Any_Hop_Skip_Requires_Compromise`	A
	helpers †	`Forward_Parent`, `Hop_Redeem_Propagates`, `Sender_Channel_Redeemed`, `Root_First`, `Next_Is_Child`, `Route_Redeemed_Gen`, `Route_Redeemed_No_Compromise`	A

however: the offer, the invoice and the `Route` fact carry no amount, so we make no end-to-end value claim.

a) Non-vacuity: Safety lemmas are useless if the model cannot run at all, so we also checked `exists-trace` witnesses. `Multihop_Routing_Reachable` shows an HTLC routed from the sender over two forwarders to the receiver, who fulfils it. `Sanity_Honest_Payment_Completes` and `Sanity_Atomic_Payment_Completes` show payments over one and over two forwarders that settle end to end, with no key compromise and no refund: every forwarder claims its incoming HTLC and the sender settles. The second witness satisfies the premises of Lemma 1. `Distinct_Parties_Configuration` shows three channels among four different parties, with no key compromise and no dispute. `Sanity_Compromised_Intermediary_Can_Lose` shows a compromised forwarder whose outgoing HTLC is redeemed while its incoming HTLC is refunded, so the premises of Lemmas 2 and 3 are satisfiable.

D. Timing and Liveness

Our model has three liveness restrictions (Sec. IV-B): T1 is `RedeemBeforeTimeout`, T2 is `OutgoingTimesOutFirst` and T3 is `IntermediaryMustClaim`. The order of a hop's two timeouts is not assumed separately: `ForwardTimeGap` is a lemma, derived from T2 and verified.

Concrete block heights are treated in two auxiliary models of `lightning_aux.spthy`. With `-D=CLTV`, a positive CLTV delta (restriction `DeltaPositive`) leaves every forwarder a non-empty claim window: a preimage revealed before the outgoing deadline is revealed before the incoming one (`Claim_Window_Nonempty`), also over two staggered hops (`Staggered_Path_Safe`). With `-D=CLOCK`, channels carry concrete deadlines on a block clock, and nine lemmas (six all-traces, three exists-trace) cover the channel and HTLC lifecycle, e.g., `No_HTLC_After_Close`; we return to two of them below.

The four-rule model `lightning_aux.spthy -D=TIMEOUT` has no ordering restriction at all. There TAMARIN finds the early-timeout race (`Early_Timeout_Race`, exists-trace, 6 steps): a forwarder's incoming HTLC times out and is refunded, and its outgoing HTLC is redeemed after that timeout. In `lightning.spthy`, the race is excluded for every forwarder that is not compromised before its incoming timeout:

```
lemma Timeout_Race_Blocked:
  "not (Ex P Prev Next Up ptrIn ptrOut y #l #i #j #k.
      HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #l
```

TABLE III

NECESSITY OF THE LIVENESS RESTRICTIONS (`make necessity`: `lightning.spthy` WITH `-D=NECESSITY` AND ONE RESTRICTION REMOVED PER COLUMN, 600 S PER CHECK). VERIFIED (n): PROVED IN n STEPS; NO RESULT: NEITHER A PROOF NOR A COUNTEREXAMPLE WITHIN 600 S, FROM WHICH WE DRAW NO CONCLUSION. THE COUNTEREXAMPLE `Necessity_IMC_Honest_Intermediary_Loses` IS CHECKED ONLY WITH T3 REMOVED.

Lemma	T1 removed `-D=NO_RBT`	T2 removed `-D=NO_OTF`	T3 removed `-D=NO_IMC`
`ForwardTimeGap`	verified (6)	no result	verified (6)
`Timeout_Race_Blocked`	verified (4)	verified (4)	verified (4)
`Intermediary_Never_Loses_Under_Liveness` (Lemma 2)	verified (4)	verified (4)	no result
`Intermediary_Never_Loses_Strong` (Lemma 3)	verified (4)	verified (4)	no result
`Payment_Atomicity_Under_Liveness` (Lemma 1)	verified (10)	no result	no result
`Necessity_IMC_Honest_Intermediary_Loses` (exists-trace)	–	–	verified (28)

```
    & TimedOut(ptrIn)     @ #k
    & Refunded(Up, ptrIn, y)  @ #j
    & Redeemed(ptrOut, y) @ #i
    & #k < #i
    & not (Ex #c. LtkCompromised(P) @ #c & #c < #k))"
```

This is a two-sided check: the race is reachable without ordering restrictions and blocked in `lightning.spthy`. `Timeout_Race_Blocked` stays verified when any single one of T1, T2 and T3 is removed (Table III).

We removed each liveness restriction in turn (`make necessity`) and re-ran the five lemmas that may depend on them (Table III). Without T3, TAMARIN finds a counterexample (exists-trace, 28 steps):

```
lemma Necessity_IMC_Honest_Intermediary_Loses:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

No key is compromised, `F1` forwards, `R` redeems the outgoing HTLC of `F1`, `F1` never claims its incoming HTLC, and the HTLC of `S` to `F1` is refunded: the honest intermediary `F1` loses, which violates Lemmas 2 and 3. (The direct proof attempts of these two lemmas without T3 gave no result within 600 s; the counterexample is this separate exists-trace lemma.) T2 only orders the two timeouts; it does not force the intermediary to claim its incoming HTLC. That claim is exactly T3, and no ordering rule can force an action. So T3 is load-bearing for intermediary safety.

The two ordering restrictions are not needed in this way. T1 is not needed by any of the five lemmas: all of them stay verified without it. T2 is not needed for intermediary safety: Lemmas 2 and 3 stay verified without it. We draw no conclusion from the cells of Table III without a result; in particular, we do not claim that `ForwardTimeGap` or Lemma 1 needs T2, or that Lemma 1 needs T3.

a) Liveness as Restrictions: Two of our three liveness assumptions, T2 and T3, are TAMARIN restrictions of the form *if A happens, then B happens* (T1 only orders a redemption before a timeout); in T3, for instance, B is the redemption of the forwarder's incoming HTLC. A restriction removes every trace that violates it, including traces in which B is impossible rather than merely late. A lemma proved under T3 therefore says nothing about runs in which an honest forwarder *cannot* claim in time, which is where real timing attacks live. Likewise, T2 removes every trace in which an honest forwarder's incoming HTLC times out while its outgoing HTLC has not timed out earlier. We make the scope of this assumption explicit.

Which results depend on it. Removing T3 breaks intermediary safety (the counterexample above); Table III shows which of the five liveness lemmas are still proved without each restriction. The 24 lemmas of `make general`, namely the bypass theorem for routes of any length (Sec. VI-B), its helpers and 16 lemmas of `make verify`, among them `Skipped_Requires_Compromise` and `Refund_Requires_Timeout`, are proved with all three liveness restrictions removed. The wormhole trace `Wormhole_Fee_Theft_Reachable` is verified with every restriction in place, so the restrictions do not hide it. All other all-traces lemmas of `make verify`, including the bypass theorems for positions 1 to 6, are proved with all three restrictions in place; apart from the five lemmas of Table III, we have not checked which of them still hold without them.

A claim window exists, in a separate model. The Clock model (`lightning_aux.spthy -D=CLOCK`) has no liveness restriction. It proves `Claim_Window_Exists`: if a forwarder's outgoing HTLC is redeemed and its incoming HTLC is refunded, the redemption happens at a strictly lower block height than the refund, so the forwarder had a non-empty window in which to claim. It also proves `Loss_Implies_Skipped_Claim`: a forwarder that loses in this way never claimed its incoming HTLC. The Clock model is not composed with `lightning.spthy`, so it supports T3 but does not discharge it.

Alternatives. A more faithful treatment would replace the restrictions by the progress semantics of Backes et al. [23]. Roughly, honest processes that can continue without input from the adversary eventually do, so traces in which the forwarder cannot act are kept instead of pruned. TAMARIN 1.10 implements this for SAPIC+ processes (option `translation-progress`), so the routing layer would have to be written as SAPIC+ processes. A cheaper first step is to turn T1, T2 and T3 into premises of only the lemmas that need them, so that they no longer prune traces for every other lemma. We leave both to future work (Sec. VIII).

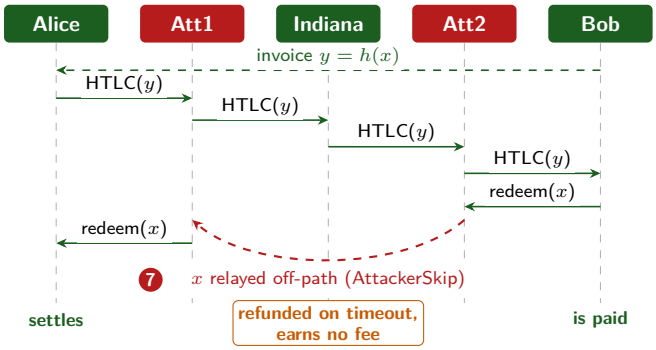


Fig. 4. Wormhole (`Wormhole_Fee_Theft_Reachable`). Att1 and Att2 are compromised (`AttackerSkip` requires this) and not adjacent; Alice, Indiana and Bob are honest. Att2 relays x to Att1 around Indiana, so Alice settles and Bob is paid, while Indiana's outgoing HTLC is refunded and Indiana earns no fee. The lemma fixes no order of its events; we draw them in causal order.

E. Griefing and the Wormhole

`Refund_Possible` shows a griefing trace: the sender offers an HTLC against an invoice, nobody forwards it, and it is refunded, so the payment never settles; no key is compromised. Since a refund requires a timeout (`Refund_Requires_Timeout`), the capital stays locked until then. This witness refunds the sender, not an intermediary. We report it as reachable griefing (denial of service), not as a new attack; it belongs to group 6 of Table II, the circled 6 in Fig. 3b.

The wormhole attack [6] is an instance of the agent-skipping attack formalised symbolically by [22]. It is a known attack, so we do not claim it as new. First, after the linear single-spend fix (Sec. V-B), it is a soundness check: the model must still admit this behaviour, or the fix would be too strong. Second, the model shows who can mount the attack. The wormhole is the attacker rule `AttackerSkip` (Sec. V-A3): two compromised parties on the same payment pass the preimage x from the exit party's outgoing HTLC to the entry party's outgoing HTLC, around the hops between them.

`Wormhole_Fee_Theft_Reachable` (exists-trace, verified by replaying a saved 116-step proof) is the five-party trace of Fig. 4, marked by the circled 7. Bob creates an invoice for Alice, and the HTLC on $y = h(x)$ is locked hop by hop from Alice through Att1, Indiana and Att2 to Bob. Bob redeems the HTLC of Att2 with x and is paid. `AttackerSkip` then relays x from Att2 to Att1, off the path, and Att1 claims the HTLC of Alice, so Alice settles. Indiana's outgoing HTLC to Att2 is refunded on timeout, and Indiana earns no fee. The lemma requires that Alice, Bob and Indiana are never compromised; in the trace, Att1 and Att2 are compromised, as `AttackerSkip` requires. An honest hop is thus bypassed although the payment settles (full statement in Appendix A). Conversely, a skip requires a key compromise:

Lemma 6 (Skips Require Compromise)**.** *Every wormhole skip happens after some party's long-term key was compromised.*

```
lemma Skipped_Requires_Compromise [reuse]:
  "All p x #i. Skipped(p, x) @ #i ==> Ex P #c. LtkCompromised(P)
      ↪ @ #c & #c < #i"
```

Lemma 6 is also re-proved without the liveness restrictions (`make general`). It does not exclude the wormhole; it says that the wormhole needs a compromise. Sec. VI-A proves, for the forwarder at each position 1 to 6 of a route (Indiana is at position 2), that if the sender settles while that forwarder's outgoing HTLC is refunded, some party's key was compromised (group 9 of Table II; under T1 to T3, with no assumption on the number of offers). Sec. VI-B extends this to routes of any length, for payments whose hash is offered once, without the liveness restrictions. Neither result says which party was compromised, or when. Sec. VI-E shows, on a fixed route of `lightning_aux.spthy`, that per-hop locks rule out the attack around an honest hop when the sender is honest (`No_Wormhole_Around_Honest_Hop`), whereas a shared hash lock admits it (`Wormhole_Around_Honest_Hop`).

VI. Bypass Resistance for Routes of Any Length

The wormhole of Sec. V-E lets a payment settle while an honest forwarder between two compromised parties is bypassed: its outgoing HTLC is refunded and it earns no fee. We now show that in our model a bypass requires key compromise: for the forwarder at each of the positions 1 to 6 of a route, and, for payments whose hash is offered on a single channel, for a forwarder at any position of a route of any length. Secs. VI-A to VI-D concern the main theory `lightning.spthy`; there is no separate N-hop theory.

HTLC offers between channel peers are not network messages in our model. When tracing a forwarded message, Tamarin treats every party as a potential sender, and with the offer as a public, signed message, the analysis of routes of unbounded length did not terminate. We therefore swapped the public message for a private, single-use fact, `Route`, which each application of the one generic rule `Forward_HTLC` consumes and re-creates for the next hop (Sec. V-A2); what this takes away from the adversary is stated in Sec. VIII.

A. Bypass Requires Compromise

The only rule that produces a `Fulfilled` fact for an HTLC that was not redeemed is `AttackerSkip`. It requires the two parties at the ends of the skip, `ExitAtt` and `EntryAtt`, to be compromised, and it records `Skipped(ptrSkip,x)`; `Fulfilled_Source` states that every `Fulfilled` fact used in a settlement stems from an earlier redemption or skip. Every skip follows a compromise (Lemma 6).

A forwarder is *bypassed* when the payment settles at the sender while the forwarder's own hop is refunded; `Intermediary_Timeout_Refund` records such a refund as `RefundHop(ptrIn,ptrOut,y)`. For the forwarders at the

first six positions of the sender's route we prove, in the full model with T1, T2 and T3 in place:

Theorem 1 (Bypass Resistance at Positions 1 to 6)**.** *Let $1 \leq k \leq 6$. If a sender settles a payment with preimage x on its own channel, the HTLC on $h(x)$ is forwarded hop by hop from that channel by forwarders $F_1, \ldots, F_k$, and the hop of F_k is refunded, then some party's long-term key was compromised.*

For $k = 1$ and $k = 2$, the statements are:

```
lemma First_Hop_Skip_Requires_Compromise
    ↪ [heuristic={lemmas_first}]:
  "All S F1 R pSF Next pOut x #s #f #r.
     SenderSettled(S, F1, R, pSF, x) @ #s
   & HTLCForwarded(F1, S, Next, pSF, pOut, h(x)) @ #f
   & RefundHop(pSF, pOut, h(x)) @ #r
  ==> Ex P #c. LtkCompromised(P) @ #c"
```

```
lemma Second_Hop_Skip_Requires_Compromise
    ↪ [heuristic={lemmas_first}]:
  "All S F1 R pSF x F2 p1 Next pOut #s #f1 #f2 #r.
     SenderSettled(S, F1, R, pSF, x) @ #s
   & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
   & HTLCForwarded(F2, F1, Next, p1, pOut, h(x)) @ #f2
   & RefundHop(p1, pOut, h(x)) @ #r
  ==> Ex P #c. LtkCompromised(P) @ #c"
```

For the forwarder at positions 3 to 6, the lemmas `Hop3_Skip_Requires_Compromise` to `Hop6_Skip_Requires_Compromise` state the same, with the route from the sender's channel spelled out as a chain of `HTLCForwarded` facts (Appendix A). These six theorems and their seven helper lemmas (group 9 of Table II), the 13 hop-bypass lemmas of `make verify`, are verified in the full model, with all restrictions, and need no assumption on how often a payment hash is offered.

In the wormhole trace `Wormhole_Fee_Theft_Reachable` (Fig. 4), Indiana is the second forwarder (`F1` is Att1, `F2` is Indiana): Alice settles while Indiana's outgoing HTLC is refunded, and Alice, Bob and Indiana are uncompromised. The trace satisfies the premise of `Second_Hop_Skip_Requires_Compromise`, and the compromise in the conclusion is that of Att1 and Att2, which `AttackerSkip` requires: the conclusion cannot be dropped.

B. Routes of Any Length

Each fixed-position theorem names the route up to the bypassed forwarder and therefore covers one position only. The general theorem quantifies over any forward of the payment hash instead.

Theorem 2 (Bypass Resistance at Any Position)**.** *If a sender settles a payment with preimage x on its channel p_{SF}, every HTLC on $h(x)$ is offered on p_{SF}, and some forwarder of an HTLC on $h(x)$, at any position, has its hop refunded, then some party's long-term key was compromised.*

```
lemma Any_Hop_Skip_Requires_Compromise
    ↪ [heuristic={lemmas_first}]:
  all-traces
  "∀ S F1 R pSF x Me Prev Next pIn pOut #s #f #r.
    ((((SenderSettled( S, F1, R, pSF, x ) @ #s) ∧
      (HTLCForwarded( Me, Prev, Next, pIn, pOut, h(x) ) @ #f))
    ↪ ∧
     (RefundHop( pIn, pOut, h(x) ) @ #r)) ∧
     (∀ A B p #o. (HTLCOffered( A, B, p, h(x) ) @ #o) ⇒ (p =
     ↪ pSF))) ⇒
    (∃ P #c. LtkCompromised( P ) @ #c)"
```

The last conjunct of the premise is the *single-offer premise*: every HTLC offered for $h(x)$ is offered on the sender's channel `pSF`. No position appears in the statement: `Me` is any forwarder of $h(x)$, at any distance from the sender.

C. Proof

The proof is organised in seven helper lemmas, proved in the same run. `Forward_Parent`: the incoming HTLC of every forward was offered by a sender or forwarded by the previous hop. `Root_First`: every forward of a payment hash is, or is preceded by, a forward whose incoming HTLC was offered by a sender. `Next_Is_Child`: if all offers of the hash are on one channel and `Prev` forwarded an HTLC to `Me`, every later forward of the hash is `Me`'s forward of that HTLC or comes after it. `Hop_Redeem_Propagates`: without compromise, if a forwarder's incoming HTLC is redeemed, so is its outgoing one. `Sender_Channel_Redeemed`: without compromise, if the sender settles, its own channel was redeemed. `Route_Redeemed_Gen`: without compromise and if all offers of the hash are on one channel, redemption of the incoming HTLC of a forward carries over to the incoming HTLC of every forward of the hash that does not come earlier. `Route_Redeemed_No_Compromise` combines it with `Sender_Channel_Redeemed` and `Root_First`: without compromise and under the single-offer premise, if the sender settles, the incoming HTLC of every forward of the hash is redeemed. `Root_First`, `Next_Is_Child` and `Route_Redeemed_Gen` are proved by induction over traces.

The theorem then takes 7 proof steps. Assume no compromise. By `Route_Redeemed_No_Compromise`, the bypassed forwarder's incoming HTLC `pIn` is redeemed. By `Redeemed_Source`, this redemption is either the settlement of a forward from `pIn`, which `Forward_Unique_Per_Incoming`, `SettledHop_From_Forward` and `No_Settle_And_Refund_Same_Hop` exclude given `RefundHop(pIn, pOut,h(x))`, or a fulfilment by the receiver, which `No_Fulfill_On_Forwarded_Incoming` excludes. The proof scripts of `Root_First`, `Next_Is_Child`, `Route_Redeemed_Gen`, `Route_Redeemed_No_Compromise` and of the theorem are stored in `lightning.spthy`; the other helpers are proved automatically.

a) General Mode and Trace Inclusion: The theorem is proved in a separate mode of the same theory, `make general` (`-D=GENERAL -D=NO_IMC -D=NO_OTF -D=NO_RBT -D=NECESSITY`), which enables the general lemmas and removes the three liveness restrictions T1 (`RedeemBeforeTimeout`), T2 (`OutgoingTimesOutFirst`) and T3 (`IntermediaryMustClaim`). Removing a restriction only adds traces, so every trace of the full model is a trace of this mode, and an all-traces property proved in this mode also holds in the full model. The run proves the theorem together with all 23 `[reuse]` lemmas available in this mode (24/24 verified), so no lemma proved under the

removed restrictions is reused; `ForwardTimeGap`, which is derived from T2, is not a `[reuse]` lemma in this mode. Bypass resistance for single-offer payments therefore does not depend on T1, T2 or T3.

D. Caveats

- *Single-offer premise.* Theorem 2 is proved only for payments whose hash is offered on a single channel. If the same hash is offered on several channels, several routes carry it, and the theorem says nothing about a forwarder on such a payment. The fixed-position theorems do not need the premise, because they spell out the route from the settling sender's channel; for such payments they cover positions 1 to 6 of that route.
- *Weak conclusion.* The conclusion states only that some party's long-term key was compromised at some point of the trace. It does not say which party (for instance a neighbour of the bypassed forwarder, as in the wormhole) and does not order the compromise before the settlement or the refund. The fixed-position theorems have the same conclusion; only `Skipped_Requires_Compromise` orders the compromise before the skip.
- *Liveness restrictions.* The six fixed-position theorems are proved in the full model, with T1, T2 and T3 in place, and we have not checked them without these restrictions. For a payment whose hash is offered on several channels, bypass resistance at positions 1 to 6 therefore rests on the liveness restrictions; only Theorem 2 (single-offer payments) is shown to hold without them.

E. Countermeasure: PTLC

The wormhole works because every channel of an HTLC route is locked with the same hash $h(x)$: once x is known at one point of the route, a compromised party can use it on a channel further upstream. Malavolta et al. [6] replace the shared lock by anonymous multi-hop locks, the basis of point time-locked contracts (PTLCs): every channel has its own lock, and a forwarder can compute the secret of its incoming lock only from the secret of its outgoing lock and its own tweak, chosen by the sender. We compare both designs in two small models of `lightning_aux.spthy` that differ only in how the channel secrets are derived and, accordingly, in how an honest hop obtains its incoming secret (`Hop_Settle`): `-D=PTLC` (per-hop secrets) and `-D=HTLC` (one shared secret, the baseline).

a) Model: Both models fix the route $\mathrm{S} \to \mathrm{H1} \to \mathrm{H2} \to \mathrm{H3} \to \mathrm{R}$ over channels `c0`, `c1`, `c2`, `c3`. `Sender_Setup` draws a fresh secret x and per-hop tweaks t_1, t_2, t_3 and derives the channel secrets by a hash chain $s_{in} = h(\langle s_{out}, t \rangle)$: $s_3 = x$, $s_2 = h(\langle s_3, t_3 \rangle)$, $s_1 = h(\langle s_2, t_2 \rangle)$, $s_0 = h(\langle s_1, t_1 \rangle)$; channel c_i is locked with $h(s_i)$. An honest hop that learns the secret of its outgoing channel derives the secret of its incoming channel with its tweak (`Hop_Settle`). The hash chain is a symbolic stand-in for the additive tweak of PTLCs built from adaptor signatures [6], $s_{in} = s_{out} + t$: in both, deriving a hop's incoming secret requires that hop's tweak. In the HTLC baseline, $s_0 = s_1 = s_2 = s_3 = x$, i.e., one hash lock $h(x)$ on every channel, and `Hop_Settle` passes the outgoing secret on unchanged. The two rules of the PTLC model that implement the per-hop locks are:

```
rule Sender_Setup:
    let s3 = ~x
        s2 = h(<s3, ~t3>)
        s1 = h(<s2, ~t2>)
        s0 = h(<s1, ~t1>)
    in
    [ Fr(~x), Fr(~c0), Fr(~c1), Fr(~c2), Fr(~c3), Fr(~t1),
      ↪ Fr(~t2), Fr(~t3) ]
    --[ Pay($S, $H1, $H2, $H3, $R, ~c0, ~c1, ~c2, ~c3),
        Neq($S, $H1), Neq($S, $H2), Neq($S, $H3), Neq($S, $R),
        Neq($H1, $H2), Neq($H2, $H3), Neq($H1, $H3),
        Neq($H1, $R), Neq($H2, $R), Neq($H3, $R) ]->
    [ !Lock(~c0, $S, $H1, h(s0)), Open(~c0), SenderWait($S, ~c0),
      !Hop($H1, ~c0, ~c1, $H2, h(s1), ~t1),
      !Hop($H2, ~c1, ~c2, $H3, h(s2), ~t2),
      !Hop($H3, ~c2, ~c3, $R,  h(s3), ~t3),
      !RecvSecret($R, ~c3, ~x),
      Out(h(s0)), Out(h(s1)), Out(h(s2)), Out(h(s3)) ]
```

```
rule Hop_Settle:
    [ Pend($H, cIn, cOut, t), ClaimInfo(cOut, sec), !Lock(cIn,
      ↪ P, $H, Lin), Open(cIn) ]
    --[ Redeemed(cIn), HopSettled($H, cIn, cOut), Eq(h(h(<sec,
      ↪ t>)), Lin) ]->
    [ ClaimInfo(cIn, h(<sec, t>)), Out(h(<sec, t>)) ]
```

The adversary is the same in both models: Dolev-Yao plus compromise of any party at any time (rule `Compromise`). A compromised hop leaks its tweak (`Leak_Tweak`); every secret used to claim a channel is published, as it would be on chain; a compromised payee may claim its channel with any value the adversary knows and that opens the lock (`Adversary_Claim`). There is no skip rule: a wormhole has to be built from adversary knowledge alone. Timing is abstracted: an open HTLC may be refunded at any time (`Refund`).

b) Results: With per-hop locks, the sender's channel `c0` is never redeemed while H2's outgoing channel `c2` is refunded, as long as S and H2 are uncompromised, whichever of H1, H3 and R are compromised (all-traces, verified, 25 steps):

```
lemma No_Wormhole_Around_Honest_Hop:
  all-traces
  "not (
Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k))"
```

With the shared hash lock, the same situation is reachable (exists-trace, verified, 12 steps):

```
lemma Wormhole_Around_Honest_Hop:
  exists-trace
  "Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k)"
```

Two control lemmas show that the PTLC result is not vacuous. `Control_Refund_Of_H2_Outgoing` (exists-trace,

6 steps): H2's outgoing HTLC can be refunded while H2 is honest, so the refund in the premise is not excluded by the model itself. `Control_Wormhole_If_H2_Compromised` (exists-trace, 18 steps): once H2 itself is compromised, the bypass is reachable with S still uncompromised, so the safety proof depends on H2 being honest. Sanity lemmas in both models show that a payment completes end to end without any compromise (`Honest_Payment_Completes`: 10 steps in PTLC, 10 in HTLC) and with both neighbours H1 and H3 of H2 compromised (`Colluders_Present_Payment_Completes`: 14 steps in PTLC, 8 in HTLC). All eight lemmas are verified by `make aux` (PTLC 5/5, HTLC 3/3).

c) Limitations: The two models are separate from `lightning.spthy`: a fixed route with three intermediaries and one honest hop under attack, no channel layer, abstract timing, and an honest `Sender_Setup` that chooses all locks and tweaks consistently; the safety lemma also assumes that the sender is never compromised. The hash chain abstracts the group operation of the adaptor-signature construction: it captures that a hop's incoming secret requires that hop's tweak, but not the algebra of [6] (an XOR encoding exhausted memory in Tamarin). We therefore claim the contrast between the two lock designs on this route, not a PTLC result for routes of any length.

VII. Related Work

Grundmann and Hartenstein [8] model-check a stepwise refinement of the Lightning Network in TLA+; model checking explores a bounded state space. Our Tamarin proofs are symbolic and unbounded: subject to the restrictions of our model (Sec. IV-B), including three liveness restrictions, they cover any number of parties, channels, channel updates and payments over routes of any length, and bypass resistance holds at every position of a route for single-offer payments (Sec. VI). [9] presents a formally verified Lightning Network.

Kiayias and Litos [24] give a composable security treatment of the Lightning Network: they define an ideal functionality for a payment network and prove, in the universal composability framework, that the Lightning protocol realises it. Their guarantee is computational and composable, and their proof is by hand. Ours are symbolic but machine-checked by Tamarin; they include bypass resistance for routes of any length (for single-offer payments), and they contrast the wormhole under HTLCs with its absence under PTLC-style locks on a fixed route.

Malavolta et al. [6] describe the wormhole attack on HTLC-based multi-hop payments and propose anonymous multi-hop locks as a countermeasure; their instantiation with adaptor signatures is the basis of PTLCs. Adaptor signatures also underlie the generalised channels of Aumayr et al. [7], built from limited blockchain scripts. Our main model reproduces the wormhole (`Wormhole_Fee_Theft_Reachable`, Sec. V-E), and our auxiliary models check the countermeasure symbolically on a fixed route (Sec. VI-E), with a hash chain in place of the adaptor-signature algebra.

Smith et al. [22] formalise agent-skipping attacks on message-forwarding protocols; the wormhole is an instance of such an attack. Our bypass theorems (Sec. VI) state the corresponding security property for Lightning routing: a forwarder at positions 1 to 6 of a route, or at any position for single-offer payments, can be skipped only if some long-term key is compromised.

Backes et al. [23] reason about liveness in cryptographic protocols with a progress semantics, in which honest processes that can continue without input from the adversary eventually do, and apply it to fair exchange. We impose liveness as trace restrictions instead (Sec. V-D).

VIII. Limitations

- *Abstract timing, liveness as restrictions.* The main model has no clock: `Open_Timeout` lets a channel time out at any point allowed by the restrictions, and T1, T2 and T3 (the last two only for forwarders not compromised before their incoming HTLC times out) stand in for CLTV deadlines and timely claiming. As restrictions, they remove every trace that violates them, including traces in which an uncompromised forwarder could not have claimed in time; results proved under them say nothing about such runs (Sec. V-D). Only the 24 lemmas of `make general`, among them Theorem 2, are proved with all three removed; the necessity runs remove one at a time, and T3 is necessary for intermediary safety (Sec. V-D). Concrete block heights appear only in the auxiliary CLOCK and CLTV models, which are not composed with `lightning.spthy`. The progress semantics of Backes et al. [23] would keep the traces in which an honest forwarder cannot act instead of pruning them. Tamarin 1.10 implements it for Sapic+ processes (option `translation-progress`), so the routing layer would have to be written as Sapic+ processes; we leave this to future work.
- *Single-offer premise.* The any-length theorem assumes that every HTLC for the payment hash is offered on the sender's channel; when a hash is offered more than once, only the fixed-position theorems (positions 1 to 6) apply, and these are proved under T1, T2 and T3. The conclusion of all bypass theorems says that some long-term key was compromised, not whose or when.
- *No end-to-end amounts.* Each `Forward_HTLC` reads its incoming amount, fee and outgoing amount from the network and requires the incoming amount to be the outgoing amount plus the fee, but the offer, the invoice and the `Route` fact carry no amount. Amounts are therefore not chained along the path, and we make no end-to-end claims about value, e.g., that the receiver obtains the invoiced amount or that the sender pays exactly the path fees.

- *HTLCs are not part of the commitment transactions.* The channel layer and the routing layer share the theory, the adversary and the channel topology (HTLCs are added only on opened channels, `HTLC_On_Opened_Channel`), but HTLC outputs are not folded into the commitment states, and settling or refunding an HTLC does not update the channel state. That each HTLC is resolved at most once is assumed as the restriction `OneResolutionPerHTLC`, not derived from the channel layer; we do not claim a composition theorem. A channel carries at most one pending HTLC at a time (`Mutex(ptr)`).
- *Routing abstraction.* HTLC offers travel as the private linear fact `Route`, not as network messages, so the network adversary cannot inject, replay or modify offers between peers; it chooses amounts and fees, controls the delivery of invoices and can compromise keys. The adversary never learns a preimage (`Preimage_Never_Known`), so in the main model the wormhole comes from the explicit rule `AttackerSkip`, which requires the parties at both ends of the skip to be compromised, rather than from adversary knowledge. (The auxiliary PTLC and HTLC models have no such rule.)
- *No onion routing, no privacy.* Every forwarder knows its predecessor, its successor and the payment hash, and the next hop is any party with which the forwarder shares a channel. We make no privacy claims.
- *PTLC model.* The countermeasure is checked only on the fixed route $\mathrm{S} \to \mathrm{H1} \to \mathrm{H2} \to \mathrm{H3} \to \mathrm{R}$, with honest lock set-up, an honest sender and a hash chain in place of the adaptor-signature algebra (Sec. VI-E).

IX. Conclusion

We presented one Tamarin theory of the Lightning Network that covers the channel layer and multi-hop HTLC routing over routes of any length, against a Dolev-Yao adversary that can also compromise long-term keys. We proved that only cheating or compromised parties are punished, that settlements are authenticated by the receiver's invoice and that preimages stay secret; under three liveness restrictions, an uncompromised intermediary never has its outgoing HTLC redeemed while its incoming one is refunded, and payments are atomic in runs without key compromise. The assumption that intermediaries claim in time is necessary: without it, Tamarin finds a run without any compromise in which an honest intermediary loses. The wormhole is a Tamarin-checked trace in which two compromised parties bypass an honest one, which is refunded and earns no fee. Conversely, a bypass requires some key compromise: under the liveness restrictions at positions 1 to 6 of any route, and, for payments whose hash is offered on a single channel, at every position even without them. On a fixed route, per-hop locks in the style of PTLCs rule out the bypass of an honest hop when the sender is honest, whereas a shared hash lock admits it. Next, we want to replace the liveness restrictions by the progress semantics of [23], so that runs in which an honest party cannot act are analysed rather than pruned, and to derive `OneResolutionPerHTLC` from the channel layer instead of assuming it.

Artifact Availability

The artifact is available as supplementary material. It consists of `lightning.spthy` (main model: channel layer, multi-hop routing and the wormhole; 37 rules), `lightning_aux.spthy` (five auxiliary models, selected by `-D=PTLC`, `-D=HTLC`, `-D=CLOCK`, `-D=CLTV` and `-D=TIMEOUT`) and a `Makefile`. `make all` runs, in this order, `make verify` (`lightning.spthy`, full model: 55/55 lemmas verified), `make general` (Sec. VI: 24/24), `make aux` (`lightning_aux.spthy`: 21/21) and `make necessity` (Sec. V-D); Table I lists what each target proves. All results were obtained with Tamarin 1.10 and reproduced on a laptop with a 6 GB heap (`MEM=6G`); `make verify-one` proves one lemma per run for machines with less memory. Tamarin 1.10 reports the Maude version used, 3.2, as unsupported; all results above were obtained with it. Every run also reports that Tamarin's derivation checks, part of its well-formedness checks of the model, timed out; this warning does not concern the proofs.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," p. 21260, 2008.

[2] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments," 2016.

[3] (2026) Lightning Network Statistics | 1ML – Lightning Network Search and Analysis Engine – Bitcoin mainnet. [Online]. Available: https://web.archive.org/web/20260810134919/https://1ml.com/statistics

[4] "Lightning/bolts," Lightning Network, 2026. [Online]. Available: https://github.com/lightning/bolts

[5] S. Tikhomirov, P. Moreno-Sanchez, and M. Maffei, "A Quantitative Analysis of Security, Anonymity and Scalability for the Lightning Network," in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2020, pp. 387–396.

[6] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, "Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability," in *Proceedings 2019 Network and Distributed System Security Symposium*. Internet Society, 2019.

[7] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, "Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures," in *Advances in Cryptology – ASIACRYPT 2021*, M. Tibouchi and H. Wang, Eds. Springer International Publishing, 2021, vol. 13091, pp. 635–664.

[8] M. Grundmann and H. Hartenstein, "Security of the lightning network: Model checking a stepwise refinement with TLA+," in *Integrated Formal Methods*, F. Damiani and M. Farrell, Eds. Springer Nature Switzerland, 2026, pp. 313–335.

[9] G. , R. Stefański, and O. S. T. Litos, "A Formally Verified Lightning Network," in *Financial Cryptography and Data Security*, C. Garman and P. Moreno-Sanchez, Eds. Springer Nature Switzerland, 2026, pp. 3–20.

[10] D. Dolev and A. Yao, "On the security of public key protocols," vol. 29, no. 2, pp. 198–208, 1983.

[11] B. Blanchet, "A computationally sound mechanized prover for security protocols," in *2006 IEEE Symposium on Security and Privacy (S&P'06)*, 2006, pp. 15 pp.–154.

[12] G. Barthe, B. Grégoire, S. Heraud, and S. Z. Béguelin, "Computer-Aided Security Proofs for the Working Cryptographer," in *Advances in Cryptology – CRYPTO 2011*, ser. Lecture Notes in Computer Science, P. Rogaway, Ed. Springer, 2011, pp. 71–90.
[13] D. Baelde, S. Delaune, C. Jacomme, A. Koutsos, and S. Moreau, "An Interactive Prover for Protocol Verification in the Computational Model," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 537–554.
[14] S. Jeanteur, L. Kovács, M. Maffei, and M. Rawson, "CryptoVampire: Automated Reasoning for the Complete Symbolic Attacker Cryptographic Model." IEEE Computer Society, 2024, pp. 3165–3183.
[15] B. Blanchet, "An Efficient Cryptographic Protocol Verifier Based on Prolog Rules," in *14th IEEE Computer Security Foundations Workshop (CSFW-14)*. IEEE Computer Society, 2001, pp. 82–96.
[16] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, N. Sharygina and H. Veith, Eds. Springer, 2013, pp. 696–701.
[17] B. Lipp, B. Blanchet, and K. Bhargavan, "A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019, pp. 231–246.
[18] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A Comprehensive Symbolic Analysis of TLS 1.3," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. Association for Computing Machinery, 2017, pp. 1773–1788.
[19] G. Andresen, "BIP 11: M-of-N Standard Transactions," 2011-10, 2011. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0011.mediawiki
[20] E. Lombrozo, J. Lau, and P. Wuille, "BIP 141: Segregated witness (consensus layer)," 2015-12, 2015. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki
[21] P. McCorry, S. Bakshi, I. Bentov, S. Meiklejohn, and A. Miller, "Pisa: Arbitration Outsourcing for State Channels," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, ser. AFT '19. Association for Computing Machinery, 2019, pp. 16–30.
[22] Z. Smith, H. Jonker, S. Mauw, and H. Lee. (2022) Modelling Agent-Skipping Attacks in Message Forwarding Protocols.
[23] M. Backes, J. Dreier, S. Kremer, and R. Künnemann, "A Novel Approach for Reasoning about Liveness in Cryptographic Protocols and its Application to Fair Exchange," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017.
[24] A. Kiayias and O. S. T. Litos, "A Composable Security Treatment of the Lightning Network," in *2020 IEEE 33rd Computer Security Foundations Symposium (CSF)*, 2020.

Appendix A
Lemma Catalogue

This appendix renders every lemma of Table II in natural language and gives its exact TAMARIN statement, so that the table can be checked against the theories. We omit two kinds of statements: that of `Skipped_Requires_Compromise`, which is printed as Lemma 6, and those of the seven helper lemmas of the general theorem, which we list by name. The last subsection covers the lemmas of the auxiliary models.

Unless stated otherwise, every lemma below is verified in `lightning.spthy`, the main model (channel layer, multi-hop routing in which one generic rule `Forward_HTLC` serves every hop of a route of any length, and the wormhole rule `AttackerSkip`). Lemmas of the auxiliary models are verified in `lightning_aux.spthy`, one model per preprocessor flag (`-D=PTLC`, `-D=HTLC`, `-D=CLOCK`, `-D=CLTV`, `-D=TIMEOUT`). Every listing is copied verbatim from these files, with the proof script omitted. The subsections follow the groups of Table II: groups 1 to 9 first, then group 0 (channel layer and keys). We use the same naming convention as the table: A for an all-traces safety property, E for an exists-trace witness (reachability or non-vacuity).

All results were obtained with TAMARIN 1.10 and are reproduced by `make all`. `make verify` proves all 55 lemmas of the full model, i.e., with all nine restrictions in place, including the liveness restrictions T1, T2 and T3 (32 all-traces safety lemmas, 13 hop-bypass lemmas, 9 exists-trace lemmas found automatically, and `Wormhole_Fee_Theft_Reachable`, replayed from a saved 116-step proof). `make general` proves the 24 lemmas of the any-length theorem in the model without the three liveness restrictions. `make aux` proves the 21 lemmas of `lightning_aux.spthy`, and `make necessity` removes one liveness restriction at a time (Table III).

Running a single check. A lemma is checked on its own by calling TAMARIN on its file with `--prove=` followed by the lemma name, and with the flags of its mode; an auxiliary model is proved as a whole with its flag and `--prove`. For example, for a lemma of `make verify`, a lemma of the general mode and the CLTV model:

```
make all
tamarin-prover lightning.spthy --prove=ForwardTimeGap
tamarin-prover lightning.spthy -D=GENERAL -D=NO_IMC \
  -D=NO_OTF -D=NO_RBT -D=NECESSITY \
  --prove=Any_Hop_Skip_Requires_Compromise
tamarin-prover lightning_aux.spthy -D=CLTV --prove
```

A single check uses the preceding lemmas marked `[reuse]` as assumptions; `make verify` and `make general` prove those lemmas too. `Wormhole_Fee_Theft_Reachable` is checked with `--lemma=` in place of `--prove=`, which replays the proof saved in the file. The TIMEOUT model also needs `--heuristic=c` and `--stop-on-trace=seqdfs`. If memory is short, `make verify-one` checks the lemmas of `make verify` one per run, and `MEM` sets the heap limit of TAMARIN (6 GB by default).

Action facts. Throughout, action facts carry their intended meaning from Sec. V-A; $y = h(x)$ is a payment hash and `ptr` a channel.

- `InvoiceCreated(R, S, y)`: receiver `R` issued an invoice for y to sender `S`. The invoice, the tuple of the tag `'invoice'`, `R`, `S` and y, is signed by `R` and carries no amount.
- `SecretPreimage(R, x)` and `PreimageReleased(R, x)`: `R` generated the preimage x, and `R` released it.
- `HTLCOffered(S, F1, ptr, y)`: the sender `S` offered an HTLC on y to its first forwarder `F1` on channel `ptr`. `OfferFor(S, R, ptr, y)` annotates the same step with the receiver `R` of the invoice.
- `HTLCAdded(ptr, y)`: an HTLC on y was added to channel `ptr`, by an offer or by a forward.
- `HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y)`: forwarder `Me`, holding an incoming HTLC from `Prev` on `ptrIn`, added an outgoing HTLC to `Next` on

`ptrOut` (one rule for every hop). `FeeCharged(Me, y, %fee)` is the fee `Me` charges at that forward.

- `HTLCFulfilled(R, Prev, ptr, y)`: the receiver redeemed its incoming HTLC.
- `IntermediaryClaimed(Me, Prev, ptrIn, x)` and `FeeEarned(Me, ptrIn, ptrOut, y, %fee)`: forwarder `Me` redeemed its incoming HTLC with x, and earned its fee.
- `SenderSettled(S, F1, R, ptr, x)`: the sender settled its HTLC with x.
- `Redeemed(ptr, y)` and `Refunded(Up, ptr, y)`: the HTLC on `ptr` was redeemed; or it was refunded, after its timeout, to `Up`, the party that had offered it.
- `TimedOut(ptr)`: the HTLC timeout of channel `ptr` was reached.
- `SettledHop(ptrIn, ptrOut, y)` and `RefundHop(ptrIn, ptrOut, y)`: annotations; the hop from `ptrIn` to `ptrOut` settled, or had its outgoing HTLC refunded.
- `UsedFulfilled(ptr, x)`: annotation; a settlement consumed the fulfilment of the HTLC on `ptr`.
- `FulfilledProduced(ptr, x)`: a fulfilment for `ptr` was produced by an intermediary settlement or by a skip.
- `Skipped(ptr, x)`: annotation of `AttackerSkip`; compromised parties relayed x onto `ptr` (wormhole).
- `ChannelOpen(A, B, ptr)`: `A` and `B` opened channel `ptr`.
- `KeyGen(A, ltk)` and `LtkCompromised(A)`: `A` generated its long-term key `ltk`; the long-term key of `A` was compromised.
- $K(x)$: the adversary knows the term x.

The action facts of the channel layer are explained in Appendix A-J.

A. Invoice and Preimage (Group 1)

a) `Invoice_Authenticates_Settlement` (A): The sender can settle an HTLC only if the payment hash corresponds to an invoice the receiver actually issued, at a strictly earlier time; the only way around this is for the receiver's long-term key to have been compromised (strictly earlier as well). In short, a settlement is legitimate only when a genuine invoice backs it; a settlement without such an invoice is attributable to a compromised receiver.

```
lemma Invoice_Authenticates_Settlement:
    "All S F1 R ptr x #i.
        SenderSettled(S, F1, R, ptr, x) @ #i
     ==> (Ex #j. InvoiceCreated(R, S, h(x)) @ #j & #j < #i)
       | (Ex #r. LtkCompromised(R) @ #r & #r < #i)"
```

b) `Forged_Invoice_Requires_Key_Compromise` (E): Reachability witness. There is a trace in which the sender settles an HTLC for receiver `R` although `R` never created an invoice for $h(x)$; the long-term key of `R` is compromised in this trace, and no forwarding takes place. Together with `Invoice_Authenticates_Settlement`, which requires such a compromise to precede the settlement, this shows that the compromise case of that lemma is not vacuous.

```
lemma Forged_Invoice_Requires_Key_Compromise:
    exists-trace
    "Ex S F1 R ptr x #i #r.
        SenderSettled(S, F1, R, ptr, x) @ #i
     & LtkCompromised(R) @ #r
     & not (Ex #j. InvoiceCreated(R, S, h(x)) @ #j)
     & not (Ex Me Prev Next pi po y #u. HTLCForwarded(Me, Prev,
      ↪ Next, pi, po, y) @ #u)"
```

c) `Invoice_Has_Secret_Preimage` (A): Every invoice is created together with the receiver's secret preimage of its hash.

```
lemma Invoice_Has_Secret_Preimage [reuse]:
    "All R S x #i.
        InvoiceCreated(R, S, h(x)) @ #i
     ==> SecretPreimage(R, x) @ #i"
```

d) `Preimage_Secret_Until_Released` (A): As long as the receiver holds a secret preimage and has not released it, the adversary does not know it: the first moment the adversary can learn the preimage is strictly after the receiver has released it. Formally, if the receiver generated a secret preimage and the adversary later learns it, then the receiver must have released that preimage at a strictly earlier point in the trace. In this model the preimage is never sent to the network: fulfilments travel between hops as state facts, and `AttackerSkip` relays them without the adversary learning x. Hence `Preimage_Never_Known` holds, and this lemma holds trivially.

```
lemma Preimage_Secret_Until_Released:
    "All R x #i #j.
        SecretPreimage(R, x) @ #i
     & K(x) @ #j
     & #i < #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #j"
```

e) `Preimage_Never_Known` (A): The adversary never learns a receiver's preimage (x is never output in this model); `Preimage_Secret_Until_Released` therefore holds trivially.

```
lemma Preimage_Never_Known:
    "All R x #i. SecretPreimage(R, x) @ #i ==> not (Ex #j. K(x)
     ↪ @ #j)"
```

f) `Invoice_Released_Once` (A): The receiver releases a given preimage at most once: if the same receiver releases the same preimage at two times in a trace, those two times coincide, i.e., there is a single release event. This holds because `Receiver_Fulfill_HTLC` consumes the linear fact `ReceiverInv(R, S, x)`, which `Receiver_Create_Invoice` creates once per fresh preimage x.

```
lemma Invoice_Released_Once [reuse]:
    "All R x #i #j.
        PreimageReleased(R, x) @ #i
     & PreimageReleased(R, x) @ #j
     ==> #i = #j"
```

B. Lock Causality (Group 2)

a) `Forward_Requires_Incoming` (A): A forwarder adds an outgoing HTLC only if an HTLC on the same hash was added earlier to its incoming channel, either offered to it by a sender or forwarded to it by its predecessor. The lemma covers every hop of a route of any length and needs no honesty assumption. It is Lemma 4.

```
lemma Forward_Requires_Incoming [use_induction]:
    "All Me Prev Next ptrIn ptrOut y #i.
        HTLCForwarded(Me, Prev, Next, ptrIn, ptrOut, y) @ #i
     ==> (Ex S #j. HTLCOffered(S, Me, ptrIn, y) @ #j & #j < #i)
       | (Ex P0 pin0 #j. HTLCForwarded(Prev, P0, Me, pin0,
      ↪ ptrIn, y) @ #j & #j < #i)"
```

b) `HTLC_On_Opened_Channel` (A): An HTLC is only ever added on a channel that has been opened beforehand: whenever an HTLC is added on a channel, two parties opened that channel at a strictly earlier time. `HTLCAdded` is emitted by `Sender_Offer_HTLC` and `Forward_HTLC`.

```
lemma HTLC_On_Opened_Channel [reuse, use_induction]:
    "All ptr y #i.
        HTLCAdded(ptr, y) @ #i
     ==> Ex X Y #j. ChannelOpen(X, Y, ptr) @ #j & #j < #i"
```

c) `Offer_Requires_Invoice` (A): An HTLC is offered for receiver `R` only against an invoice that `R` created earlier, or the long-term key of `R` is compromised at some point of the trace.

```
lemma Offer_Requires_Invoice:
    "All S R ptr y #o.
        OfferFor(S, R, ptr, y) @ #o
     ==> (Ex #j. InvoiceCreated(R, S, y) @ #j & #j < #o)
       | (Ex #r. LtkCompromised(R) @ #r)"
```

C. Release Causality (Group 3)

a) `Claim_Requires_Release` (A): An intermediary can claim its incoming HTLC with a preimage only after (strictly later than) the receiver has released that preimage: the release of the preimage is a prerequisite for any intermediary settlement.

```
lemma Claim_Requires_Release [use_induction]:
    "All MidP InP ptrIn R x #i #j.
        IntermediaryClaimed(MidP, InP, ptrIn, x) @ #i
     & SecretPreimage(R, x) @ #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```

b) `Settle_Requires_Receiver_Release` (A): When the sender settles an HTLC on a genuine invoice, the receiver must have released the corresponding preimage strictly earlier: a settlement never precedes the receiver's release of the preimage.

```
lemma Settle_Requires_Receiver_Release:
    "All S F1 R ptr x #i #j.
        SenderSettled(S, F1, R, ptr, x) @ #i
     & InvoiceCreated(R, S, h(x)) @ #j
     ==> Ex #k. PreimageReleased(R, x) @ #k & #k < #i"
```

c) `Redeem_Requires_Receiver_Release` (A): Every redeemed HTLC on y comes with a receiver's release of a preimage x with $h(x) = y$.

```
lemma Redeem_Requires_Receiver_Release [use_induction]:
    "All ptr y #i.
        Redeemed(ptr, y) @ #i
     ==> Ex R x #k. PreimageReleased(R, x) @ #k & h(x) = y"
```

d) `Fulfilled_Requires_Release` (A): Every fulfilment produced by an intermediary settlement or a skip is preceded by a receiver's release of x.

```
lemma Fulfilled_Requires_Release [reuse, use_induction,
      ↪ heuristic=S]:
    "All ptr x #i.
        FulfilledProduced(ptr, x) @ #i
     ==> Ex R #k. PreimageReleased(R, x) @ #k & #k < #i"
```

e) `Fulfilled_Source` (A): A fulfilment consumed by a settlement stems from an earlier redemption of that HTLC or from an earlier skip.

```
lemma Fulfilled_Source [reuse]:
    "All p x #i. UsedFulfilled(p, x) @ #i
     ==> (Ex #j. Redeemed(p, h(x)) @ #j & #j < #i)
       | (Ex #j. Skipped(p, x) @ #j & #j < #i)"
```

D. Atomicity (Group 4)

a) `Payment_Atomicity_Under_Liveness` (A): If two forwarders `F1` and `F2` forward consecutive HTLCs on the same hash y (the outgoing channel of `F1` is the incoming channel of `F2`), the HTLC on the outgoing channel of `F2` is redeemed, and no party's long-term key is ever compromised, then the HTLC on the incoming channel of `F1` is never refunded: in a run without any key compromise, once the payment is redeemed downstream of two forwarders, the party upstream of them cannot be refunded. The lemma is proved in the full model, i.e., under the liveness restrictions; it also verifies with T1 removed (Table III). It is Lemma 1.

```
lemma Payment_Atomicity_Under_Liveness:
    "All F1 F2 Prev1 Next1 Prev2 Next2 ptrSF1 ptrF1F2 ptrF2R y
     ↪ #i #j #k.
        HTLCForwarded(F1, Prev1, Next1, ptrSF1, ptrF1F2, y) @ #i
     & HTLCForwarded(F2, Prev2, Next2, ptrF1F2, ptrF2R, y) @ #j
     & Redeemed(ptrF2R, y) @ #k
     & (not Ex P #l. LtkCompromised(P) @ #l)
     ==> not (Ex Up #m. Refunded(Up, ptrSF1, y) @ #m)"
```

b) `Settle_Excludes_Sender_Refund` (A): If the sender has settled an HTLC with a preimage, then that same HTLC is never refunded: settlement and refund of the sender's HTLC are mutually exclusive.

```
lemma Settle_Excludes_Sender_Refund [heuristic={sender_pending}]:
    "All S F1 R ptr x #i.
        SenderSettled(S, F1, R, ptr, x) @ #i
     ==> not (Ex #j. Refunded(S, ptr, h(x)) @ #j)"
```

c) `OneOutcomePerHTLC` (A): No HTLC is both redeemed and refunded. Both outcomes emit the action `Old` for the same channel and hash, which the restriction `OneResolutionPerHTLC` allows only once.

```
lemma OneOutcomePerHTLC [reuse]:
    "All Up ptr y #i #j.
       Redeemed(ptr, y) @ #i & Refunded(Up, ptr, y) @ #j ==> F"
```

d) `Multihop_Routing_Reachable` (E): Non-vacuity witness. The routing phase of a payment over two forwarders is reachable: the sender offers, both forwarders forward and the receiver fulfils, with no other offer or forward, at most one HTLC per channel, and no intermediary claim or sender settlement.

```
lemma Multihop_Routing_Reachable:
    exists-trace
    "Ex S F1 F2 R p0 p1 p2 x #t2 #t3 #t4 #t5.
        HTLCOffered(S, F1, p0, h(x)) @ #t2
     & HTLCForwarded(F1, S, F2, p0, p1, h(x)) @ #t3
     & HTLCForwarded(F2, F1, R, p1, p2, h(x)) @ #t4
     & HTLCFulfilled(R, F2, p2, h(x)) @ #t5
     & #t2 < #t3 & #t3 < #t4 & #t4 < #t5
     & not (Ex A B p y #u. HTLCOffered(A, B, p, y) @ #u & not
     ↪ (A = S & B = F1))
     & not (Ex Me Prev Next pi po y #u. HTLCForwarded(Me, Prev,
     ↪ Next, pi, po, y) @ #u
              & not (Me = F1 & Prev = S & Next = F2)
              & not (Me = F2 & Prev = F1 & Next = R))
```

```
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
      & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @
      ↪ #e)
      & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

e) `Sanity_Honest_Payment_Completes` (E): Non-vacuity witness. An honest payment over one forwarder completes end to end: the offer, the forward, the fulfilment, the forwarder's claim and the sender's settlement occur, with no key compromise, at most one HTLC per channel, exactly this claim and this settlement, and no refund.

```
lemma Sanity_Honest_Payment_Completes:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #c #s.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & IntermediaryClaimed(F1, S, p0, x) @ #c
    & SenderSettled(S, F1, R, p0, x) @ #s
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & (All Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e
      ↪ ==> #e = #c)
    & (All A B C q z #e. SenderSettled(A, B, C, q, z) @ #e ==>
      ↪ #e = #s)
    & not (Ex U q z #e. Refunded(U, q, z) @ #e)"
```

f) `Sanity_Atomic_Payment_Completes` (E): Non-vacuity witness. An honest payment over two forwarders completes end to end: the offer, both forwards, the fulfilment, both intermediary claims and the sender's settlement occur, with no key compromise, at most one HTLC per channel, exactly these claims and this settlement, and no refund. It shows that the premise of `Payment_Atomicity_Under_Liveness` is satisfiable.

```
lemma Sanity_Atomic_Payment_Completes:
  exists-trace
  "Ex S F1 F2 R p0 p1 p2 x #o #a #b #r #c1 #c2 #s.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, F2, p0, p1, h(x)) @ #a
    & HTLCForwarded(F2, F1, R, p1, p2, h(x)) @ #b
    & HTLCFulfilled(R, F2, p2, h(x)) @ #r
    & IntermediaryClaimed(F2, F1, p1, x) @ #c2
    & IntermediaryClaimed(F1, S, p0, x) @ #c1
    & SenderSettled(S, F1, R, p0, x) @ #s
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & (All Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e
      ↪ ==> (#e = #c1 | #e = #c2))
    & (All A B C q z #e. SenderSettled(A, B, C, q, z) @ #e ==>
      ↪ #e = #s)
    & not (Ex U q z #e. Refunded(U, q, z) @ #e)"
```

g) `Distinct_Parties_Configuration` (E): Non-vacuity witness. Three channels among four distinct parties can be opened with no compromise, update, publication, close, cheat or forward, and with at most one HTLC per channel.

```
lemma Distinct_Parties_Configuration:
    exists-trace
    "Ex S F1 F2 R ptrSF1 ptrF1F2 ptrF2R #t1 #t2 #t3.
        ChannelOpen(S, F1, ptrSF1) @ #t1
      & ChannelOpen(F1, F2, ptrF1F2) @ #t2
      & ChannelOpen(F2, R, ptrF2R) @ #t3
      & not (S = F1) & not (S = F2) & not (S = R)
      & not (F1 = F2) & not (F1 = R) & not (F2 = R)
      & not (Ex P #i. LtkCompromised(P) @ #i)
      & not (Ex P Q p n #j. StateUpdate(P, Q, p, n) @ #j)
      & not (Ex P n #k. PublishState(P, n) @ #k)
      & not (Ex P Q p n #l. Close(P, Q, p, n) @ #l)
      & not (Ex P n #m. Cheat(P, n) @ #m)
      & not (Ex Me Prev Next pi po y #n. HTLCForwarded(Me, Prev,
      ↪ Next, pi, po, y) @ #n)
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

E. Fees (Group 5)

Amounts are local to a hop. `Forward_HTLC` receives the incoming amount, the fee and the outgoing amount as inputs and emits an `Eq` action stating that the incoming amount is the outgoing amount plus the fee, which the `Equality` restriction enforces; on every hop, the incoming amount therefore equals the outgoing amount plus the fee by construction (Sec. V-A4). The offer, the invoice and the route carry no amount, so amounts are not chained along the path and the model makes no end-to-end value statement. The fee guarantee is the following lemma.

a) `Fee_Only_On_Successful_Forward` (A): A forwarder whose key is never compromised earns its fee only after its outgoing HTLC was redeemed and after it forwarded on that hop. It is Lemma 5.

```
lemma Fee_Only_On_Successful_Forward
      ↪ [heuristic={route_info_first}]:
    "All Me ptrIn ptrOut y %fee #i.
        FeeEarned(Me, ptrIn, ptrOut, y, %fee) @ #i
      & not (Ex #c. LtkCompromised(Me) @ #c)
     ==> (Ex #j. Redeemed(ptrOut, y) @ #j & #j < #i)
       & (Ex Prev Next #f. HTLCForwarded(Me, Prev, Next, ptrIn,
      ↪ ptrOut, y) @ #f & #f < #i)"
```

F. Refund and Griefing (Group 6)

a) `Refund_Requires_Timeout` (A): Every refund of an HTLC is preceded by the timeout of its channel: an HTLC is refunded only after that channel has timed out.

```
lemma Refund_Requires_Timeout [reuse]:
    "All Up ptr y #i.
        Refunded(Up, ptr, y) @ #i
     ==> Ex #j. TimedOut(ptr) @ #j & #j < #i"
```

b) `Loss_Requires_Inaction` (A): If a forwarder's outgoing HTLC is redeemed while its incoming HTLC is refunded, then its incoming HTLC is never redeemed. An intermediary therefore loses funds only through its own inaction, by failing to redeem its incoming HTLC before it times out, and never because the same HTLC was both redeemed and refunded (the two outcomes are exclusive).

```
lemma Loss_Requires_Inaction:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> not (Ex #l. Redeemed(ptrIn, y) @ #l)"
```

c) `Refund_Possible` (E): Reachability witness. A payment can be refunded: the receiver issues an invoice, the sender offers an HTLC on its hash, and the sender's HTLC is refunded, with no settlement, no key compromise, no forwarding, and at most one HTLC per channel. Since a refund requires a timeout (`Refund_Requires_Timeout`), the sender's funds stay locked until then; Sec. V-E reports this trace as reachable griefing of the sender.

```
lemma Refund_Possible:
    exists-trace
    "Ex S F1 R p y #i #j #k.
        InvoiceCreated(R, S, y) @ #j
```

```
      & HTLCOffered(S, F1, p, y) @ #k
      & Refunded(S, p, y) @ #i
      & #j < #k & #k < #i
      & not (Ex A MidP B p2 x #l. SenderSettled(A, MidP, B, p2,
      ↪ x) @ #l)
      & not (Ex P #r. LtkCompromised(P) @ #r)
      & not (Ex Me Prev Next pi po y2 #t1. HTLCForwarded(Me,
      ↪ Prev, Next, pi, po, y2) @ #t1)
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

G. Wormhole (Group 7)

a) `Wormhole_Fee_Theft_Reachable` (E): Reachability witness (attack). There is a trace on the route Alice → Att1 → Indiana → Att2 → Bob in which Bob issues the invoice and redeems his incoming HTLC and Alice settles her HTLC, yet the outgoing HTLC of the intermediary Indiana is refunded and Indiana earns no fee. Alice, Bob and Indiana are never compromised, Att1 and Att2 are distinct parties, and every channel carries at most one HTLC. This is the wormhole [6] of Fig. 4: the preimage is relayed around Indiana by rule `AttackerSkip`, which only two compromised parties on the same payment can apply. TAMARIN does not find this trace automatically: `make verify` replays a saved 116-step proof. The statement, in the pretty-printed notation of TAMARIN, is shown in Fig. 5. In the PTLC model of `lightning_aux.spthy`, the corresponding wormhole around an honest hop is unreachable (`No_Wormhole_Around_Honest_Hop`), while the HTLC baseline reaches it (`Wormhole_Around_Honest_Hop`); see Appendix A-K.

b) `Skipped_Requires_Compromise` (A): Every wormhole skip happens after some party's long-term key was compromised. Its statement is Lemma 6 in Sec. V-E. It is also re-proved without the liveness restrictions (`make general`).

H. Intermediary Safety and Timing (Group 8)

1) Intermediary Safety:

a) `Intermediary_Never_Loses_Under_Liveness` (A): If an intermediary's outgoing HTLC is redeemed while its incoming HTLC is refunded, then its long-term key was compromised. Equivalently, an honest intermediary whose outgoing HTLC is redeemed never has its incoming HTLC refunded. The stronger lemma below places the compromise before the incoming timeout. This lemma is Lemma 2; it also verifies with T1 or T2 removed (Table III).

```
lemma Intermediary_Never_Loses_Under_Liveness:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l. LtkCompromised(P) @ #l"
```

b) `Intermediary_Never_Loses_Strong` (A): A forwarder whose outgoing HTLC is redeemed while its incoming HTLC is refunded was compromised before its incoming HTLC timed out. It is Lemma 3; it also verifies with T1 or T2 removed (Table III).

```
lemma Intermediary_Never_Loses_Strong:
    "All P Prev Next Up ptrIn ptrOut y #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #i
      & Redeemed(ptrOut, y) @ #j
      & Refunded(Up, ptrIn, y) @ #k
     ==> Ex #l #t. LtkCompromised(P) @ #l & TimedOut(ptrIn) @ #t
      ↪ & #l < #t"
```

c) `Sanity_Compromised_Intermediary_Can_Lose` (E): Non-vacuity witness for `Intermediary_Never_Loses_Under_Liveness`: a compromised forwarder can lose (its outgoing HTLC is fulfilled while its incoming HTLC is refunded), with at most one HTLC per channel, no intermediary claim and no sender settlement.

```
lemma Sanity_Compromised_Intermediary_Can_Lose:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u #k.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & LtkCompromised(F1) @ #k
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

d) `Necessity_IMC_Honest_Intermediary_Loses` (E): Checked with `-D=NO_IMC` and `-D=NECESSITY` (`make necessity`, 28 steps). Without T3 an honest forwarder can lose: there is a trace in which its outgoing HTLC is fulfilled by the receiver while its incoming HTLC is refunded to the sender, with no key compromise, no claim and no settlement. Hence T3 is necessary for intermediary safety (Sec. V-D).

```
lemma Necessity_IMC_Honest_Intermediary_Loses:
  exists-trace
  "Ex S F1 R p0 p1 x #o #a #r #u.
      HTLCOffered(S, F1, p0, h(x)) @ #o
    & HTLCForwarded(F1, S, R, p0, p1, h(x)) @ #a
    & HTLCFulfilled(R, F1, p1, h(x)) @ #r
    & Refunded(S, p0, h(x)) @ #u
    & not (Ex P #k. LtkCompromised(P) @ #k)
    & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
      ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))
    & not (Ex Me P q z #e. IntermediaryClaimed(Me, P, q, z) @ #e)
    & not (Ex A B C q z #e. SenderSettled(A, B, C, q, z) @ #e)"
```

2) Timing:

a) `Timeout_Race_Blocked` (A): No intermediary that is not compromised before its incoming timeout has its incoming HTLC refunded when that timeout happens strictly before its outgoing HTLC is redeemed. The lemma also verifies with any one of T1, T2 and T3 removed (4 steps each, Table III), so it does not depend on any single liveness restriction. In the abstract TIMEOUT model of `lightning_aux.spthy`, where either channel may time out at any point and the outgoing HTLC may be redeemed at any time, the race is reachable (`Early_Timeout_Race`, E, 6 steps; Appendix A-K).

```
lemma Timeout_Race_Blocked:
    "not (Ex P Prev Next Up ptrIn ptrOut y #l #i #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #l
      & TimedOut(ptrIn)     @ #k
      & Refunded(Up, ptrIn, y)  @ #j
      & Redeemed(ptrOut, y) @ #i
      & #k < #i
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k))"
```

```
lemma Wormhole_Fee_Theft_Reachable:
  exists-trace
  "∃ Alice Bob Att1 Att2 Indiana ptrAliceAtt1 ptrAtt1Indiana
     ptrIndianaAtt2 ptrAtt2Bob x #t1 #t2 #t3 #t4 #t5 #t6 #t7 #t8.
    ((((((((((((((InvoiceCreated( Bob, Alice, h(x) ) @ #t1) ∧
                  (HTLCOffered( Alice, Att1, ptrAliceAtt1, h(x) ) @ #t2)) ∧
                 (HTLCForwarded( Att1, Alice, Indiana, ptrAliceAtt1, ptrAtt1Indiana,
                                 h(x)
                  ) @ #t3)) ∧
                (HTLCForwarded( Indiana, Att1, Att2, ptrAtt1Indiana,
                                ptrIndianaAtt2, h(x)
                 ) @ #t4)) ∧
               (HTLCForwarded( Att2, Indiana, Bob, ptrIndianaAtt2, ptrAtt2Bob,
                               h(x)
                ) @ #t5)) ∧
              (HTLCFulfilled( Bob, Att2, ptrAtt2Bob, h(x) ) @ #t6)) ∧
             (SenderSettled( Alice, Att1, Bob, ptrAliceAtt1, x ) @ #t7)) ∧
            (Refunded( Indiana, ptrIndianaAtt2, h(x) ) @ #t8)) ∧
           (¬(Indiana = Alice))) ∧
          (¬(Indiana = Bob))) ∧
         (¬(Att1 = Att2))) ∧
        (¬(∃ #c. LtkCompromised( Alice ) @ #c))) ∧
       (¬(∃ #c. LtkCompromised( Bob ) @ #c))) ∧
      (¬(∃ #c. LtkCompromised( Indiana ) @ #c))) ∧
     (¬(∃ ptr1 ptr2 yy %fee #k.
         FeeEarned( Indiana, ptr1, ptr2, yy, %fee ) @ #k))) ∧
    (¬(∃ ptr y1 y2 #m #n.
        ((HTLCAdded( ptr, y1 ) @ #m) ∧ (HTLCAdded( ptr, y2 ) @ #n)) ∧
        (¬(#m = #n))))"
```

Fig. 5. Statement of `Wormhole_Fee_Theft_Reachable` in `lightning.spthy`, in the pretty-printed notation of TAMARIN (exists-trace; verified by replaying a saved 116-step proof).

b) `ForwardTimeGap` (A): If both channels of a forward time out and the forwarder is not compromised before its incoming timeout, the outgoing HTLC times out strictly before the incoming one. The order of a hop's two timeouts is thus not assumed: it is this lemma, derived from T2 (Sec. IV-B). It also verifies with T1 or T3 removed (Table III). With `-D=NECESSITY`, the same statement is declared without `[reuse]`, so that no other proof uses it in the necessity runs and in the general mode.

```
lemma ForwardTimeGap [reuse]:
    "All P Prev Next ptrIn ptrOut y #f #j #k.
        HTLCForwarded(P, Prev, Next, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrOut) @ #j
      & TimedOut(ptrIn)  @ #k
      & not (Ex #c. LtkCompromised(P) @ #c & #c < #k)
     ==> #j < #k"
```

I. Hop Bypass (Group 9)

If the payment settles at the sender while a forwarder's own hop is refunded (its outgoing HTLC is refunded, `RefundHop`), some party's key was compromised. The conclusion does not say which party, or when. The lemmas for positions 1 to 6 are proved by `make verify`, in the full model under T1 to T3, and need no single-offer premise; `Any_Hop_Skip_Requires_Compromise` covers any position on a route of any length under a single-offer premise (every offer on $h(x)$ uses the sender's channel `pSF`) and is proved by `make general`. Sec. VI discusses these results.

1) Positions 1 to 6:

a) `First_Hop_Skip_Requires_Compromise` (A): The first forwarder cannot be bypassed without a compromise: if the sender settles its HTLC on channel `pSF` while the hop of its first forwarder `F1` from `pSF` is refunded, some party's key was compromised.

```
lemma First_Hop_Skip_Requires_Compromise
      ↪ [heuristic={lemmas_first}]:
    "All S F1 R pSF Next pOut x #s #f #r.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, Next, pSF, pOut, h(x)) @ #f
      & RefundHop(pSF, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

b) `Second_Hop_Skip_Requires_Compromise` (A): The second forwarder (the position of Indiana in `Wormhole_Fee_Theft_Reachable`) cannot be bypassed without a compromise.

```
lemma Second_Hop_Skip_Requires_Compromise
      ↪ [heuristic={lemmas_first}]:
    "All S F1 R pSF x F2 p1 Next pOut #s #f1 #f2 #r.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, Next, p1, pOut, h(x)) @ #f2
      & RefundHop(p1, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

c) `Hop3_Skip_Requires_Compromise` (A): The forwarder at position 3 cannot be bypassed without a compromise. The statement adds a third forward to the chain of `Second_Hop_Skip_Requires_Compromise`; the statements for positions 4 to 6 each add one more.

```
lemma Hop3_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 p1 p2 #s #r #f1 #f2 #f3.
        SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, Next, p2, pOut, h(x)) @ #f3
      & RefundHop(p2, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

d) Hop4_Skip_Requires_Compromise (A): The forwarder at position 4 cannot be bypassed without a compromise.

```
lemma Hop4_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 p1 p2 p3 #s #r #f1 #f2
     ↪ #f3 #f4.
       SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
      & HTLCForwarded(F4, F3, Next, p3, pOut, h(x)) @ #f4
      & RefundHop(p3, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

e) Hop5_Skip_Requires_Compromise (A): The forwarder at position 5 cannot be bypassed without a compromise.

```
lemma Hop5_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 F5 p1 p2 p3 p4 #s #r
     ↪ #f1 #f2 #f3 #f4 #f5.
       SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
      & HTLCForwarded(F4, F3, F5, p3, p4, h(x)) @ #f4
      & HTLCForwarded(F5, F4, Next, p4, pOut, h(x)) @ #f5
      & RefundHop(p4, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

f) Hop6_Skip_Requires_Compromise (A): The forwarder at position 6 cannot be bypassed without a compromise.

```
lemma Hop6_Skip_Requires_Compromise [heuristic={lemmas_first}]:
    "All S R x pSF pOut Next F1 F2 F3 F4 F5 F6 p1 p2 p3 p4 p5 #s
     ↪ #r #f1 #f2 #f3 #f4 #f5 #f6.
       SenderSettled(S, F1, R, pSF, x) @ #s
      & HTLCForwarded(F1, S, F2, pSF, p1, h(x)) @ #f1
      & HTLCForwarded(F2, F1, F3, p1, p2, h(x)) @ #f2
      & HTLCForwarded(F3, F2, F4, p2, p3, h(x)) @ #f3
      & HTLCForwarded(F4, F3, F5, p3, p4, h(x)) @ #f4
      & HTLCForwarded(F5, F4, F6, p4, p5, h(x)) @ #f5
      & HTLCForwarded(F6, F5, Next, p5, pOut, h(x)) @ #f6
      & RefundHop(p5, pOut, h(x)) @ #r
     ==> Ex P #c. LtkCompromised(P) @ #c"
```

2) Helper Lemmas: The hop-bypass lemmas rest on seven helper lemmas, all of kind A and declared `[reuse]`. All seven are also re-proved in `make general`.

a) Forward_Unique_Per_Incoming (A): An incoming HTLC is forwarded at most once: two forwards of the same hash from the same incoming channel are the same event.

```
lemma Forward_Unique_Per_Incoming [reuse,
      ↪ heuristic={route_first}]:
    "All M1 P1 N1 M2 P2 N2 pIn o1 o2 y #i #j.
       HTLCForwarded(M1, P1, N1, pIn, o1, y) @ #i
      & HTLCForwarded(M2, P2, N2, pIn, o2, y) @ #j
     ==> #i = #j"
```

b) SettledHop_From_Forward (A): A hop settles only if it was forwarded earlier, from the same incoming to the same outgoing channel.

```
lemma SettledHop_From_Forward [reuse, heuristic={route_first}]:
    "All pIn pOut y #i. SettledHop(pIn, pOut, y) @ #i
     ==> Ex M P N #j. HTLCForwarded(M, P, N, pIn, pOut, y) @ #j
      ↪ & #j < #i"
```

c) No_Settle_And_Refund_Same_Hop (A): No hop both settles and has its outgoing HTLC refunded.

```
lemma No_Settle_And_Refund_Same_Hop [reuse,
      ↪ heuristic={route_first}]:
    "All pIn pOut y #i #j. SettledHop(pIn, pOut, y) @ #i &
     ↪ RefundHop(pIn, pOut, y) @ #j ==> F"
```

d) No_Fulfill_On_Forwarded_Incoming (A): The HTLC on a channel is never both fulfilled by a receiver and forwarded onwards.

```
lemma No_Fulfill_On_Forwarded_Incoming [reuse,
      ↪ heuristic={route_first}]:
    "All R Q M P N p o y #i #j.
       HTLCFulfilled(R, Q, p, y) @ #i & HTLCForwarded(M, P, N,
      ↪ p, o, y) @ #j ==> F"
```

e) Redeemed_Source (A): An HTLC is redeemed only by the settlement of the hop for which it is the incoming HTLC, or by the receiver's fulfilment, in the same step.

```
lemma Redeemed_Source [reuse]:
    "All p y #i. Redeemed(p, y) @ #i
     ==> (Ex o. SettledHop(p, o, y) @ #i)
       | (Ex R Q. HTLCFulfilled(R, Q, p, y) @ #i)"
```

f) SenderSettled_Uses_Fulfilled (A): A sender settlement consumes the fulfilment of the sender's own channel.

```
lemma SenderSettled_Uses_Fulfilled [reuse]:
    "All S F1 R p x #i. SenderSettled(S, F1, R, p, x) @ #i ==>
     ↪ UsedFulfilled(p, x) @ #i"
```

g) SettledHop_Uses_Outgoing (A): A hop settlement consumes the fulfilment of the hop's outgoing channel, with a preimage of the hop's hash.

```
lemma SettledHop_Uses_Outgoing [reuse]:
    "All pIn pOut y #i. SettledHop(pIn, pOut, y) @ #i
     ==> Ex x. UsedFulfilled(pOut, x) @ #i & h(x) = y"
```

3) Routes of Any Length (General Mode):

a) Any_Hop_Skip_Requires_Compromise (A): Proved by `make general` in 7 steps, with the flags shown at the start of this appendix. A single-offer payment that settles while any forwarder's own hop is refunded requires a compromise. The lemma is proved in the model without the three liveness restrictions; removing restrictions only adds traces, so the statement also holds in the full model. It is Theorem 2; the statement is written in the pretty-printed notation of Tamarin.

```
lemma Any_Hop_Skip_Requires_Compromise
      ↪ [heuristic={lemmas_first}]:
  all-traces
  "∀ S F1 R pSF x Me Prev Next pIn pOut #s #f #r.
    ((((SenderSettled( S, F1, R, pSF, x ) @ #s) ∧
       (HTLCForwarded( Me, Prev, Next, pIn, pOut, h(x) ) @ #f))
      ↪ ∧
      (RefundHop( pIn, pOut, h(x) ) @ #r)) ∧
     (∀ A B p #o. (HTLCOffered( A, B, p, h(x) ) @ #o) ⇒ (p =
      ↪ pSF))) ⇒
    (∃ P #c. LtkCompromised( P ) @ #c)"
```

The general theorem uses a chain of seven helper lemmas, all of kind A and proved only in this mode; we list them by name.

- `Forward_Parent`: the statement of `Forward_Requires_Incoming`, declared `[reuse]` for this mode.
- `Hop_Redeem_Propagates`: with no compromise, if the incoming HTLC of a forwarder is redeemed, so is its outgoing HTLC.
- `Sender_Channel_Redeemed`: with no compromise, a sender settlement implies that the sender's channel was redeemed.

- `Root_First`: every forward either forwards an offered HTLC or is preceded by a forward that does.
- `Next_Is_Child`: with a single offer, every forward on the hash that follows a forward to `Me` is, or comes after, the forward of `Me` from that channel.
- `Route_Redeemed_Gen`: with a single offer and no compromise, if the incoming HTLC of a forward is redeemed, so is the incoming HTLC of every forward on the same hash at the same time or later.
- `Route_Redeemed_No_Compromise`: with a single offer and no compromise, if the sender settles, the incoming HTLC of every forwarder on the hash is redeemed.

`make general` proves 24 lemmas: the theorem, these seven helpers, and 16 lemmas of `make verify` re-proved in the same mode, namely the seven helper lemmas of the hop-bypass family above, `OneOutcomePerHTLC`, `Ltk_Known_Implies_Compromised`, `Invoice_Released_Once`, `Invoice_Has_Secret_Preimage`, `HTLC_On_Opened_Channel`, `Fulfilled_Requires_Release`, `Refund_Requires_Timeout`, `Fulfilled_Source` and `Skipped_Requires_Compromise`.

J. Channel Layer and Keys (Group 0)

These lemmas concern the channel layer (Sec. V-A1) and the long-term keys. Their action facts read as follows.

- `StateUpdate(A, B, ptr, n)`: the channel moved to state `n` after both parties revoked their previous state.
- `NewStateBuilt(A, B, ptr, n, fA, fB)`: state `n` with balances `fA` and `fB` was built, at opening or by `Update_State`.
- `FundsLocked(A, B, ptr)`: the funds of both parties were locked when the channel opened.
- `PublishState(A, n)`: `A` published its commitment of state `n`.
- `Cheat(A, n)` and `Punish(B, A, n)`: `A` published its revoked state `n`, and `B` punished this and took all funds of the channel.
- `Close(A, B, ptr, n)`: the channel was closed cooperatively in state `n`.
- `OnChainDelayedProduce(A, n)` and `OnChainInstantProduce(A, n)`: an on-chain transaction of state `n` created a delayed (CSV-locked) or an instant output for `A`.
- `ClaimFunds(A, n)` and `FundsSettled(A, n)`: `A` claimed its delayed output after the CSV delay, or received its instant output.
- `SettledTx(A, tx)`: the transaction `tx` paid `A`.

a) `state_update` (A): A channel is updated only after the same two parties opened it.

```
lemma state_update [use_induction]:
    "All A B ptr n #i.
       StateUpdate(A, B, ptr, n) @ #i
     ==> Ex #j. ChannelOpen(A, B, ptr) @ #j & #j < #i"
```

b) `Funds_Locked_Before_Update` (A): A channel is updated only after the funds of both parties were locked in it.

```
lemma Funds_Locked_Before_Update [use_induction]:
    "All A B ptr n #i.
       StateUpdate(A, B, ptr, n) @ #i
     ==> Ex #j. FundsLocked(A, B, ptr) @ #j & #j < #i"
```

c) `Update_Requires_Negotiation` (A): A channel moves to a new state only after both parties built that state, with its balances.

```
lemma Update_Requires_Negotiation [use_induction]:
    "All A B ptr n #i.
         StateUpdate(A, B, ptr, n) @ #i
     ==> Ex fA fB #j. NewStateBuilt(A, B, ptr, n, fA, fB) @ #j &
      ↪ #j < #i"
```

d) `delayed_funds` (A): A party claims a delayed output of state `n` only after an on-chain transaction of state `n` created that output for it.

```
lemma delayed_funds:
    "All A n #i.
       ClaimFunds(A, n) @ #i
     ==> Ex #j. OnChainDelayedProduce(A, n) @ #j & #j < #i"
```

e) `instant_funds` (A): A party receives an instant output of state `n` only after an on-chain transaction of state `n` created that output for it.

```
lemma instant_funds:
    "All A n #i.
       FundsSettled(A, n) @ #i
     ==> Ex #j. OnChainInstantProduce(A, n) @ #j & #j < #i"
```

f) `settlement_is_traceable` (A): Every instant settlement names the transaction that pays the party, in the same step.

```
lemma settlement_is_traceable:
    "All A n #i.
       FundsSettled(A, n) @ #i
     ==> Ex tx #j. SettledTx(A, tx) @ #j & #j = #i"
```

g) `No_Punishment_Without_Cheating` (A): A party is punished for state `n` only if it published that revoked state earlier, or if its long-term key was compromised earlier.

```
lemma No_Punishment_Without_Cheating:
    "All A B n #i.
        Punish(B, A, n) @ #i
     ==> (Ex #k. Cheat(A, n) @ #k & #k < #i)
       | (Ex #j. LtkCompromised(A) @ #j & #j < #i)"
```

h) `Ltk_Known_Implies_Compromised` (A): The adversary knows a long-term key only if that key was compromised. Here `!KU(ltk)` means that the adversary can construct `ltk`.

```
lemma Ltk_Known_Implies_Compromised [reuse]:
    "All A ltk #i #j.
        KeyGen(A, ltk) @ #i
      & !KU(ltk) @ #j
     ==> Ex #k. LtkCompromised(A) @ #k"
```

i) `Protocol_execution` (E): Non-vacuity witness. A channel can be opened, a new state built and committed by an update, and the channel closed cooperatively in that state, in this order, with at most one HTLC per channel.

```
lemma Protocol_execution:
    exists-trace
    "Ex A B ptr n fA fB #i #j #k #l.
        ChannelOpen(A, B, ptr) @ #i
      & NewStateBuilt(A, B, ptr, n, fA, fB) @ #j
      & StateUpdate(A, B, ptr, n) @ #k
```

```
      & Close(A, B, ptr, n) @ #l
      & #i < #j & #j < #k & #k < #l
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
    ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

j) `Cooperative_Close_Execution` (E): Non-vacuity witness. A channel can be opened and closed cooperatively without any state update, with at most one HTLC per channel.

```
lemma Cooperative_Close_Execution:
    exists-trace
    "Ex A B ptr n #k #i.
        ChannelOpen(A, B, ptr) @ #k & Close(A, B, ptr, n) @ #i &
     ↪ #k < #i
      & not (Ex A2 B2 ptr2 n2 #j. StateUpdate(A2, B2, ptr2, n2)
     ↪ @ #j)
      & not (Ex ptr y1 y2 #m #n. HTLCAdded(ptr, y1) @ #m &
     ↪ HTLCAdded(ptr, y2) @ #n & not (#m = #n))"
```

K. Auxiliary Models

The auxiliary models of `lightning_aux.spthy` are self-contained and are not composed with `lightning.spthy` (Sec. IV-C). Table IV lists their 21 lemmas, all verified by `make aux`; we print the key statements below.

The PTLC model (`-D=PTLC`) has the fixed route S → H1 → H2 → H3 → R over the channels `c0` to `c3`. Each channel has its own secret, derived from the secret of the next channel by the hash chain $s_{in} = h(\langle s_{out}, t \rangle)$ with a fresh tweak t per hop, a symbolic stand-in for the adaptor-signature points of [6]; locks are hashes of the secrets. The HTLC baseline (`-D=HTLC`) has the same route with one secret x for every channel. In both models a compromised hop leaks its tweak, the secret used to claim a channel becomes public, and a compromised party can claim its incoming channel with any value the adversary knows. There is no skip rule: a wormhole must be built from the adversary's knowledge. The CLOCK model (`-D=CLOCK`) gives channels concrete CLTV deadlines on a block clock, lets a forwarder forward only if its outgoing deadline is strictly lower than its incoming one, and records the block heights at which HTLCs are redeemed, claimed and refunded. The CLTV model (`-D=CLTV`) registers hops whose incoming deadline exceeds the outgoing one by a positive delta (restriction `DeltaPositive`) and reveals a preimage only strictly before the outgoing deadline (restriction `FulfillInTime`). The TIMEOUT model (`-D=TIMEOUT`) has four rules and no ordering: either channel may time out at any point, and the outgoing HTLC may be redeemed at any time.

a) `No_Wormhole_Around_Honest_Hop` (A, PTLC): In the PTLC model, the channel `c0` of the sender is never redeemed while the outgoing channel `c2` of an uncompromised H2 is refunded, unless the sender is compromised. The result depends on the tweak of H2 staying secret: once H2 is compromised, the wormhole is reachable (`Control_Wormhole_If_H2_Compromised`), and the refund of the outgoing channel of an uncompromised H2 alone is reachable too (`Control_Refund_Of_H2_Outgoing`).

```
lemma No_Wormhole_Around_Honest_Hop:
  all-traces
  "not (
```

TABLE IV
LEMMAS OF THE AUXILIARY MODELS (`LIGHTNING_AUX.SPTHY`, `MAKE AUX`, 21/21 VERIFIED). A: ALL-TRACES; E: EXISTS-TRACE; STEPS: SIZE OF THE PROOF FOUND BY TAMARIN.

Flag	Lemma	Kind	Steps
`PTLC`	`Honest_Payment_Completes`	E	10
	`Colluders_Present_Payment_Completes`	E	14
	`No_Wormhole_Around_Honest_Hop`	A	25
	`Control_Refund_Of_H2_Outgoing`	E	6
	`Control_Wormhole_If_H2_Compromised`	E	18
`HTLC`	`Honest_Payment_Completes`	E	10
	`Colluders_Present_Payment_Completes`	E	8
	`Wormhole_Around_Honest_Hop`	E	12
`CLOCK`	`HTLC_Needs_Open_Channel`	A	36
	`No_HTLC_After_Close`	A	367
	`Claim_Window_Exists`	A	79
	`Transitive_Preimage_Before_Upstream_Deadline`	A	187
	`Loss_Implies_Skipped_Claim`	A	861
	`Outcome_Exclusive`	A	2
	`Redeem_Reachable`	E	7
	`Refund_Reachable`	E	7
	`Honest_Flow_Possible`	E	14
`CLTV`	`CLTV_Gap_Is_Positive`	A	2
	`Claim_Window_Nonempty`	A	2
	`Staggered_Path_Safe`	A	2
`TIMEOUT`	`Early_Timeout_Race`	E	6

```
Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k))"
```

b) `Wormhole_Around_Honest_Hop` (E, HTLC): With HTLCs, that wormhole around an uncompromised H2 is reachable, with an uncompromised sender.

```
lemma Wormhole_Around_Honest_Hop:
  exists-trace
  "Ex S H1 H2 H3 R c0 c1 c2 c3 #p #a #b.
      Pay(S, H1, H2, H3, R, c0, c1, c2, c3) @ #p
    & Redeemed(c0) @ #a
    & Refunded(c2) @ #b
    & not (Ex #k. Compromised(H2) @ #k)
    & not (Ex #k. Compromised(S) @ #k)"
```

c) `Claim_Window_Exists` (A, CLOCK): If a forwarder's outgoing HTLC is redeemed at block height `bOut` and its incoming HTLC is refunded at height `bRef`, then `bOut` is strictly lower than `bRef`: the forwarder had a non-empty window in which to claim.

```
lemma Claim_Window_Exists:
    "All F pIn pOut dIn dOut y bOut bRef #f #r #u.
        Forwarded(F, pIn, pOut, dIn, dOut, y) @ #f
      & RedeemedAt(pOut, y, bOut) @ #r
      & RefundedAt(pIn,  y, bRef) @ #u
     ==> bOut << bRef"
```

d) `Loss_Implies_Skipped_Claim` (A, CLOCK): A forwarder whose outgoing HTLC is redeemed while its incoming HTLC is refunded never claimed its incoming HTLC.

```
lemma Loss_Implies_Skipped_Claim:
    "All F pIn pOut dIn dOut y #f #r #u.
```

```
      Forwarded(F, pIn, pOut, dIn, dOut, y) @ #f
    & Redeemed(pOut, y) @ #r
    & Refunded(pIn,  y) @ #u
   ==> not (Ex b #c. ClaimedAt(pIn, y, b) @ #c)"
```

e) `Claim_Window_Nonempty` (A, CLTV): A preimage revealed before the outgoing deadline of a hop is revealed before its incoming deadline.

```
lemma Claim_Window_Nonempty [reuse]:
    "All ptrOut ptrIn d_out d_in b #i #j.
       HopRegistered(ptrOut, ptrIn, d_out, d_in) @ #i
     & Revealed(b, ptrOut, d_out) @ #j
     ==> b << d_in"
```

f) `Staggered_Path_Safe` (A, CLTV): Over two staggered hops, a preimage revealed before the tightest deadline is revealed before both upstream deadlines, so both intermediaries have a non-empty claim window.

```
lemma Staggered_Path_Safe:
    "All ptr1 ptr2 ptr2b ptr3 d1 d2 d3 b #i #j #k.
       HopRegistered(ptr1, ptr2, d1, d2) @ #i
     & HopRegistered(ptr2b, ptr3, d2, d3) @ #j
     & Revealed(b, ptr1, d1) @ #k
     ==> b << d2 & b << d3"
```

g) `Early_Timeout_Race` (E, TIMEOUT): Without any ordering of timeouts, a forwarder's incoming HTLC times out and is refunded, and its outgoing HTLC is redeemed after that timeout. `Timeout_Race_Blocked` excludes this pattern in `lightning.spthy` for forwarders that are not compromised before their incoming timeout.

```
lemma Early_Timeout_Race:
    exists-trace
    "Ex P ptrIn ptrOut y #f #k #j #i.
       ForwardHTLC(P, ptrIn, ptrOut, y) @ #f
      & TimedOut(ptrIn)     @ #k
      & Refunded(ptrIn, y)  @ #j
      & Redeemed(ptrOut, y) @ #i
      & #k < #i"
```